



---

# POLITECHNIKA POZNAŃSKA

---

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI  
Instytut Informatyki

Praca dyplomowa magisterska

## ODKRYWANIE ZALEŻNOŚCI MIĘDZY OBIEKTAMI W BAZIE DANYCH

Maria Nowicka, 151851

Promotor  
dr hab. inż. Robert Wrembel, prof. PP.

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;  
oryginał wstawiamy do wersji dla archiwum PP,  
w pozostałych kopiach wstawiamy ksero.

# Streszczenie

Ciągłość grafów pochodzenia danych (*data lineage*) ulega przerwaniu, gdy w potokach przetwarzania wykorzystywane są obiekty pośrednie — tabele tymczasowe oraz funkcje definiowane przez użytkownika. Zjawisko to, określane jako *broken lineage*, pozbawia tabele informacji o ich producencie lub konsumencie danych i utrudnia audyt oraz kontrolę jakości danych. Dotychczasowe prace modelują ten problem wyłącznie jako predykcję brakujących krawędzi; nie stawiają natomiast pytania, *które tabele* mają zerwane pochodzenie danych. Niniejsza praca wypełnia tę lukę, formułując zadanie na poziomie *węzłów* — jako ranking tabel według prawdopodobieństwa zainfekowania zjawiskiem *broken lineage*.

Celem pracy jest zbadanie, w jakim stopniu *sama topologia* grafu lineage pozwala wskazać takie tabele, bez dostępu do semantyki nazw obiektów i schematów bazy. Zjawisko *broken lineage* symulowano przez kontrolowane usuwanie zadań ze zbioru DLG-DG-23 — 18 zanonimizowanych, rzeczywistych grafów lineage ze środowiska Huawei Cloud. Porównano trzy klasy metod: heurystyki strukturalne, klasyczne klasyfikatory uczenia maszynowego oparte na cechach pozycji węzła w grafie DAG oraz autorski detektor *LineageDetector*, łączący cechy topologiczne ze scorem kompletności *peer-consistency* i kalibracją izotoniczną prawdopodobieństw. Ewaluację przeprowadzono w protokole indukcyjnym cross-graph, z kontrolą przecieku wynikającego z trywialnie izolowanych węzłów.

W wariancie uczciwym (z wykluczeniem węzłów izolowanych) *LineageDetector* osiągnął najwyższe wartości wszystkich czterech stosowanych metryk: AUC-ROC 0,771, AUC-PR 0,435, Precision@k 0,443 oraz Brier 0,106. Test kolejności par Wilcoxona, sparowany po grafach, potwierdził istotność tej przewagi nad heurystykami oraz nad metodami boostingowymi; różnica wobec Random Forest pozostała w granicach szumu. Badanie ablacyjne wskazało score kompletności jako składnik odpowiadający za przewagę detektora i ujawniło zarazem, że kalibracja izotoniczna poprawia rzetelność oszacowań prawdopodobieństwa kosztem niewielkiego pogorszenia jakości rankingu.

Skuteczność okazała się stabilna względem odsetka usuwanych zadań i utrzymała się na grafie pochodzącym spoza zbioru treningowego. Ujawniono natomiast istotne ograniczenie: model uczony wyłącznie na grafach małych traci około 0,15 AUC-ROC przy ocenie grafów o rząd wielkości większych. Główny wniosek: sama topologia grafu niesie umiarkowany, lecz użyteczny sygnał o *broken lineage* na poziomie węzłów, stanowiący punkt odniesienia (*topology-only baseline*) dla metod grafowo-semantycznych.

**Słowa kluczowe:** data lineage, broken lineage, detekcja anomalii w grafach, uczenie indukcyjne, kalibracja prawdopodobieństw.

# Abstract

The continuity of data lineage graphs is disrupted whenever processing pipelines rely on intermediate objects — temporary tables and user-defined functions. This phenomenon, known as *broken lineage*, deprives tables of information about their data producer or consumer and hampers auditing and data-quality control. Existing work models the problem exclusively as missing-edge prediction; it does not ask *which tables* have broken lineage. This thesis fills that gap by formulating the task at the *node* level — as a ranking of tables by their probability of being affected by broken lineage.

The goal is to investigate to what extent the *topology alone* of a lineage graph allows such tables to be identified, without access to the semantics of object names or database schemas. Broken lineage was simulated by the controlled removal of jobs from the DLG-DG-23 dataset — 18 anonymised, real-world lineage graphs from the Huawei Cloud environment. Three classes of methods were compared: structural heuristics, classical machine-learning classifiers based on DAG-position features, and the proposed *LineageDetector*, which combines topological features with a relational peer-consistency completeness score and isotonic probability calibration. Evaluation followed an inductive cross-graph protocol, with control for the leakage caused by trivially isolated nodes.

In the fair variant (excluding isolated nodes), LineageDetector achieved the highest value of all four metrics used: AUC-ROC 0.771, AUC-PR 0.435, Precision@k 0.443, and Brier 0.106. A Wilcoxon signed-rank test, paired across graphs, confirmed that this advantage is significant over the heuristics and over the boosting-based classifiers; the difference with respect to Random Forest remained within noise. An ablation study identified the completeness score as the component responsible for the detector’s advantage, while also revealing that isotonic calibration improves the reliability of probability estimates at the cost of a slight loss in ranking quality.

Performance proved stable with respect to the fraction of removed jobs and carried over to a graph originating outside the training set. One substantial limitation was identified: a model trained solely on small graphs loses about 0.15 AUC-ROC when evaluated on graphs an order of magnitude larger. The main conclusion: graph topology alone carries a moderate yet useful signal about broken lineage at the node level, providing a topology-only baseline for graph-semantic methods.

**Keywords:** data lineage, broken lineage, graph anomaly detection, inductive learning, probability

calibration.

# Spis treści

|                                                                 |             |
|-----------------------------------------------------------------|-------------|
| <b>Streszczenie</b>                                             | <b>iii</b>  |
| <b>Abstract</b>                                                 | <b>I</b>    |
| <b>Spis rysunków</b>                                            | <b>V</b>    |
| <b>Spis rysunków</b>                                            | <b>V</b>    |
| <b>Spis tabel</b>                                               | <b>VI</b>   |
| <b>Spis tabel</b>                                               | <b>VII</b>  |
| <b>Spis listingów</b>                                           | <b>VIII</b> |
| <b>Spis listingów</b>                                           | <b>VIII</b> |
| <b>Wykaz skrótów i oznaczeń</b>                                 | <b>IX</b>   |
| <b>1 Wstęp</b>                                                  | <b>1</b>    |
| 1.1 Cel i zakres pracy . . . . .                                | 1           |
| 1.2 Pytania badawcze . . . . .                                  | 2           |
| 1.3 Wkład własny . . . . .                                      | 2           |
| 1.4 Struktura pracy . . . . .                                   | 3           |
| <b>2 Podstawy teoretyczne i przegląd literatury</b>             | <b>4</b>    |
| 2.1 Data lineage i data provenance . . . . .                    | 4           |
| 2.1.1 Rodzaje proveniencji . . . . .                            | 4           |
| 2.2 Struktura grafu lineage . . . . .                           | 5           |
| 2.3 Niekompletność grafów lineage — broken lineage . . . . .    | 5           |
| 2.3.1 Taksonomia przyczyn . . . . .                             | 6           |
| 2.4 Narzędzia zarządzania lineage w praktyce . . . . .          | 6           |
| 2.5 Detekcja węzłów o zerwanym lineage . . . . .                | 7           |
| 2.6 Wykorzystane metody uczenia maszynowego . . . . .           | 8           |
| 2.6.1 Metody zespołowe . . . . .                                | 8           |
| 2.6.2 Kalibracja prawdopodobieństw . . . . .                    | 8           |
| 2.7 Metryki ewaluacji . . . . .                                 | 9           |
| 2.8 Systematyczny przegląd literatury . . . . .                 | 9           |
| 2.8.1 Metodyka przeglądu . . . . .                              | 9           |
| 2.8.2 Prace bazowe: metoda kontrastowa i zbiór danych . . . . . | 11          |
| 2.8.3 Metody pokrewne odkrywania zależności lineage . . . . .   | 12          |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 2.8.4    | Synteza porównawcza . . . . .                                    | 13        |
| 2.8.5    | Luka badawcza . . . . .                                          | 13        |
| 2.8.6    | Prace kontekstowe . . . . .                                      | 14        |
| <b>3</b> | <b>Problem i sposób jego rozwiązania</b>                         | <b>15</b> |
| 3.1      | Sformułowanie problemu . . . . .                                 | 15        |
| 3.2      | Zbiór danych DLG-DG-23 . . . . .                                 | 15        |
| 3.2.1    | Charakterystyka grafów . . . . .                                 | 16        |
| 3.3      | Metodyka symulacji broken lineage przez usuwanie zadań . . . . . | 16        |
| 3.3.1    | Kontrola przecieku: wykluczenie węzłów izolowanych . . . . .     | 17        |
| 3.4      | Architektura rozwiązania . . . . .                               | 18        |
| 3.4.1    | Moduł wczytywania danych . . . . .                               | 18        |
| 3.4.2    | Moduł symulacji (job_removal) . . . . .                          | 18        |
| 3.4.3    | Moduł ewaluacji (node_metrics) . . . . .                         | 18        |
| 3.5      | Zaimplementowane metody detekcji . . . . .                       | 19        |
| 3.5.1    | Heurystyki strukturalne . . . . .                                | 19        |
| 3.5.2    | Klasyczne metody uczenia maszynowego . . . . .                   | 19        |
| 3.5.3    | Autorski detektor — LineageDetector . . . . .                    | 21        |
| 3.6      | Implementacja . . . . .                                          | 21        |
| 3.6.1    | Stos technologiczny . . . . .                                    | 22        |
| 3.6.2    | Organizacja repozytorium . . . . .                               | 22        |
| 3.6.3    | Format danych i wczytywanie . . . . .                            | 23        |
| 3.6.4    | Wyznaczanie score'u peer-consistency . . . . .                   | 23        |
| 3.6.5    | Protokół indukcyjny w kodzie . . . . .                           | 24        |
| 3.6.6    | Złożoność obliczeniowa . . . . .                                 | 24        |
| 3.7      | Testy jednostkowe i odtwarzalność . . . . .                      | 24        |
| 3.7.1    | Zakres testów . . . . .                                          | 24        |
| 3.7.2    | Odtwarzalność wyników . . . . .                                  | 25        |
| 3.8      | Dostępność implementacji i pochodzenie algorytmów . . . . .      | 25        |
| <b>4</b> | <b>Ocena rozwiązania</b>                                         | <b>27</b> |
| 4.1      | Badanie wstępne: predykcja krawędzi . . . . .                    | 27        |
| 4.2      | Wyniki detekcji węzłów o zerwanym lineage . . . . .              | 27        |
| 4.3      | Wyniki w rozbiciu na poszczególne grafy . . . . .                | 28        |
| 4.4      | Istotność statystyczna różnic . . . . .                          | 30        |
| 4.5      | Badanie ablacyjne detektora . . . . .                            | 31        |
| 4.6      | Kontrola przecieku i analiza wrażliwości . . . . .               | 32        |
| 4.6.1    | Przeciek izolacji . . . . .                                      | 32        |
| 4.6.2    | Wrażliwość na odsetek usuwanych zadań . . . . .                  | 32        |
| 4.7      | Ważność cech . . . . .                                           | 33        |
| 4.8      | Warianty protokołu ewaluacji . . . . .                           | 34        |
| 4.8.1    | Protokół transduktywny . . . . .                                 | 34        |
| 4.8.2    | Generalizacja na większą skalę . . . . .                         | 35        |
| 4.8.3    | Pojedyncze zdarzenie broken lineage . . . . .                    | 35        |
| 4.9      | Test na grafie spoza zbioru DLG-DG-23 (cross-dataset) . . . . .  | 36        |
| 4.10     | Analiza wyników . . . . .                                        | 37        |
| 4.11     | Zagrożenia trafności . . . . .                                   | 38        |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 4.11.1   | Trafność konstrukcji . . . . .                          | 38        |
| 4.11.2   | Trafność wewnętrzna . . . . .                           | 39        |
| 4.11.3   | Trafność zewnętrzna . . . . .                           | 39        |
| 4.11.4   | Trafność wnioskowania statystycznego . . . . .          | 39        |
| <b>5</b> | <b>Uwagi końcowe</b>                                    | <b>40</b> |
| 5.1      | Podsumowanie . . . . .                                  | 40        |
| 5.2      | Kierunki dalszych badań . . . . .                       | 41        |
|          | <b>Literatura</b>                                       | <b>42</b> |
| <b>A</b> | <b>Instrukcja uruchomienia i zawartość repozytorium</b> | <b>45</b> |
| A.1      | Wymagania i instalacja . . . . .                        | 45        |
| A.2      | Odtworzenie wyników rozdziału 4 . . . . .               | 45        |
| A.3      | Zawartość repozytorium . . . . .                        | 46        |
| <b>B</b> | <b>Pełne wyniki eksperymentów</b>                       | <b>47</b> |
| <b>C</b> | <b>Przegląd konfiguracji testu cross-dataset</b>        | <b>51</b> |

# Spis rysunków

|     |                                                                                                                                                                                                                                           |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Fragment grafu przepływu danych: (a) graf oryginalny, (b) graf po usunięciu zadania J1. Tabele zainfekowane oznaczono kolorem czerwonym, krawędzie utracone — linią przerywaną. . . . .                                                   | 5  |
| 3.1 | Schemat potoku przetwarzania. Każda instancja ewaluacji przebiega w protokole indukcyjnym leave-one-graph-out: model trenowany jest na 17 grafach, a testowany na grafie niewidzianym. . . . .                                            | 18 |
| 4.1 | Porównanie AUC-ROC metod detekcji w wariancie uczciwym (z wykluczeniem węzłów izolowanych) i napompowanym. Metody uczone zachowują przewagę w obu wariantach; heurystyki lokalne tracą najwięcej po usunięciu przecieku izolacji. . . . . | 29 |
| 4.2 | Wrażliwość AUC-ROC detektora LineageDetector na parametr <code>removal_ratio</code> . Wynik pozostaje stabilny w całym badanym zakresie. . . . .                                                                                          | 33 |



# Spis tabel

|     |                                                                                                                                                                                                                                                                                                                                               |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Przyczyny broken lineage i ich konsekwencje dla grafu zależności. . . . .                                                                                                                                                                                                                                                                     | 6  |
| 2.2 | Wybrane narzędzia zarządzania lineage i stosowane przez nie metody akwizycji meta-<br>danych. . . . .                                                                                                                                                                                                                                         | 7  |
| 2.3 | Przebieg selekcji literatury w bazie DBLP dla zapytania " <b>data provenance</b> "OR " <b>data<br/>lineage</b> ". . . . .                                                                                                                                                                                                                     | 10 |
| 2.4 | Zestawienie porównawcze omówionych prac. Ostatni wiersz — niniejsza praca. . . . .                                                                                                                                                                                                                                                            | 13 |
| 3.1 | Charakterystyka grafów zbioru DLG-DG-23. . . . .                                                                                                                                                                                                                                                                                              | 16 |
| 3.2 | Charakterystyka symulacji broken lineage w zależności od parametru <b>removal_ratio</b> .<br>Wartości uśrednione po 18 grafach i trzech ziarnach losowości (42–44). Ostatnia ko-<br>lumna — odsetek tabel zainfekowanych, które utraciły <i>wszystkie</i> krawędzie przepływu<br>danych i są rozpoznawalne trywialną regułą izolacji. . . . . | 17 |
| 3.3 | Wektor cech topologicznych węzła (moduł <b>node_features</b> ). . . . .                                                                                                                                                                                                                                                                       | 20 |
| 3.4 | Stos technologiczny i wersje bibliotek użyte w biegu kanonicznym. . . . .                                                                                                                                                                                                                                                                     | 22 |
| 3.5 | Rozkład testów jednostkowych na moduły. . . . .                                                                                                                                                                                                                                                                                               | 25 |
| 4.1 | Średnie metryki detekcji zainfekowanych tabel per metoda (protokół indukcyjny, wa-<br>riant uczciwy), wraz z odchyleniem standardowym po ośmiu grafach wiarygodnych;<br>gwiazdka — wkład własny, najlepsza wartość w kolumnie pogrubiona. . . . .                                                                                             | 28 |
| 4.2 | AUC-ROC w rozbiciu na grafy testowe (protokół indukcyjny, wariant uczciwy). Kand. — liczba<br>tabel-kandydatów, Zainf. — liczba tabel zainfekowanych; najlepsza wartość w wierszu<br>pogrubiona. . . . .                                                                                                                                      | 29 |
| 4.3 | Istotność statystyczna różnic między LineageDetectorem a pozostałymi metodami. Test<br>kolejności par Wilcozona (dokładny, dwustronny), sparowany po 8 grafach wiarygod-<br>nych; poprawka Holma w obrębie rodziny hipotez i metryki. $\Delta$ — mediana różnicy<br>na korzyść LineageDetector. . . . .                                       | 30 |
| 4.4 | Badanie ablacyjne LineageDetector (protokół indukcyjny, wariant uczciwy, 8 grafów<br>wiarygodnych). W nawiasach — różnica względem pełnego detektora; znak dla Brier<br>score odwrócono tak, aby wartość ujemna zawsze oznaczała pogorszenie. . . . .                                                                                         | 31 |
| 4.5 | Wpływ przecieku izolacji na AUC-ROC (protokół indukcyjny). . . . .                                                                                                                                                                                                                                                                            | 32 |
| 4.6 | Wrażliwość LineageDetector na <b>removal_ratio</b> (AUC-ROC, protokół indukcyjny). . . . .                                                                                                                                                                                                                                                    | 33 |
| 4.7 | Ważność cech detektora LineageDetector mierzona metodą permutacyjną: średni spadek<br>AUC-ROC po losowym przestawieniu wartości cechy. Gwiazdka — cecha wnoszona<br>przez autorski score kompletności. . . . .                                                                                                                                | 34 |
| 4.8 | Wpływ zmiany protokołu ewaluacji na AUC-ROC. Wszystkie warianty na tych samych<br>pięciu grafach testowych (DLG6, DLG10, DLG12, DLG16, DLG18), <b>removal_ratio</b> =<br>0,10, wariant uczciwy. . . . .                                                                                                                                       | 35 |

|      |                                                                                                                                                                                                    |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.9  | Wariant pojedynczego usuniętego zadania w zestawieniu z biegiem kanonicznym (protokół indukcyjny, wariant uczciwy, 8 grafów wiarygodnych). . . . .                                                 | 36 |
| 4.10 | Wynik na zewnętrznym grafie DUT (trening wyłącznie na DLG-DG-23, wariant uczciwy). Ostatnia kolumna — AUC-ROC na dużych grafach DLG-DG-23 jako odniesienie wewnątrz zbioru. . . . .                | 37 |
| B.1  | AUC-ROC dla wszystkich 18 grafów zbioru DLG-DG-23 (wyżej — lepiej). Gwiazdka przy identyfikatorze grafu — graf odrzucony z agregacji progiem $\text{MIN\_POS} = 5$ . . . .                         | 47 |
| B.2  | AUC-PR dla wszystkich 18 grafów zbioru DLG-DG-23 (wyżej — lepiej). Gwiazdka przy identyfikatorze grafu — graf odrzucony z agregacji progiem $\text{MIN\_POS} = 5$ . . . .                          | 48 |
| B.3  | Precision@k dla wszystkich 18 grafów zbioru DLG-DG-23 (wyżej — lepiej). Gwiazdka przy identyfikatorze grafu — graf odrzucony z agregacji progiem $\text{MIN\_POS} = 5$ . . . .                     | 49 |
| B.4  | Brier score dla wszystkich 18 grafów zbioru DLG-DG-23 (niżej — lepiej). Gwiazdka przy identyfikatorze grafu — graf odrzucony z agregacji progiem $\text{MIN\_POS} = 5$ . . . .                     | 50 |
| C.1  | AUC-ROC na zewnętrznym grafie DUT w zależności od wariantu izolacji i parametru <code>removal_ratio</code> (trening na małych i średnich grafach DLG-DG-23, 25 powtórzeń na konfigurację). . . . . | 51 |

## Spis listingów

|     |                                                                                                                          |    |
|-----|--------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Struktura repozytorium (fragmenty istotne dla detekcji węzłów). . . . .                                                  | 22 |
| 3.2 | Fragment plików wejściowych <code>DLGi-node.json</code> i <code>DLGi-edge.json</code> (identyfikatory skrócone). . . . . | 23 |
| 3.3 | Rdzeń wyznaczania score'u peer-consistency (moduł <code>completeness.py</code> ). . . . .                                | 23 |
| 3.4 | Szkielet pętli leave-one-graph-out (moduł <code>run_detection_experiments.py</code> ). . . .                             | 24 |

# Wykaz skrótów i oznaczeń

## Skróty

|           |                                                                                         |
|-----------|-----------------------------------------------------------------------------------------|
| AUC-PR    | pole pod krzywą precyzja-czułość ( <i>Area Under the Precision-Recall Curve</i> )       |
| AUC-ROC   | pole pod krzywą ROC ( <i>Area Under the Receiver Operating Characteristic Curve</i> )   |
| CTE       | wspólne wyrażenie tablicowe ( <i>Common Table Expression</i> )                          |
| DAG       | skierowany graf acykliczny ( <i>Directed Acyclic Graph</i> )                            |
| DLG-DG-23 | zbiór danych grafów lineage opublikowany przez Chen et al. [10]                         |
| ETL       | proces ekstrakcji, transformacji i ładowania danych ( <i>Extract, Transform, Load</i> ) |
| GNN       | grafowa sieć neuronowa ( <i>Graph Neural Network</i> )                                  |
| HGT       | heterogeniczny transformer grafowy ( <i>Heterogeneous Graph Transformer</i> )           |
| LOGO      | protokół „jeden graf wyłączony” ( <i>leave-one-graph-out</i> )                          |
| P@k       | precyzja na $k$ czołowych pozycjach rankingu ( <i>Precision@k</i> )                     |
| PGK       | jądro grafów proveniencji ( <i>Provenance Graph Kernel</i> )                            |
| PROV-O    | ontologia proveniencji W3C ( <i>W3C Provenance Ontology</i> )                           |
| RDF       | model opisu zasobów ( <i>Resource Description Framework</i> )                           |
| R-GCN     | relacyjna grafowa sieć konwolucyjna ( <i>Relational Graph Convolutional Network</i> )   |
| RODO      | Rozporządzenie o Ochronie Danych Osobowych                                              |
| RUSBoost  | boosting z losowym podpróbkowaniem ( <i>Random Under-Sampling Boosting</i> )            |
| SLR       | systematyczny przegląd literatury ( <i>Systematic Literature Review</i> )               |
| SQL       | strukturalny język zapytań ( <i>Structured Query Language</i> )                         |
| UDF       | funkcja definiowana przez użytkownika ( <i>User-Defined Function</i> )                  |
| UUID      | uniwersalny unikatowy identyfikator ( <i>Universally Unique Identifier</i> )            |

## Oznaczenia

|                  |                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------|
| $G = (V, E)$     | graf lineage: zbiór węzłów $V$ i krawędzi $E$                                                           |
| $V_T, V_J, V_F$  | zbiory węzłów typu tabela, zadanie i pole                                                               |
| $G_{\text{obs}}$ | graf obserwowany, powstały przez usunięcie zadań z $G$                                                  |
| $s(t)$           | wynik detekcji przypisany tabeli $t$ , interpretowany jako $p(t)$ zainfekowana                          |
| $P(t), C(t)$     | zbiory peerów tabeli $t$ po stronie producenta i konsumenta                                             |
| in_df, out_df    | liczba wejściowych i wyjściowych krawędzi $DATA\_FLOW$ węzła                                            |
| $k$              | liczba czołowych pozycji rankingu branych pod uwagę; w pracy $k$ równa się liczbie tabel zainfekowanych |
| $m$              | liczba usuniętych zadań                                                                                 |
| $\alpha$         | poziom istotności statystycznej                                                                         |
| $\Delta$         | mediana różnicy między porównywanymi metodami                                                           |

# Rozdział 1

## Wstęp

Współczesne systemy bazodanowe przetwarzają dane w złożonych potokach transformacji. W środowiskach korporacyjnych dane są wielokrotnie agregowane i przekształcane przez sekwencje zapytań, a wyniki pośrednio przechowywane w tabelach tymczasowych. Tymczasowość tych obiektów sprawia, że po zakończeniu procesu przetwarzania ślad po wykonanych transformacjach zanika, a zatem niemożliwe staje się ustalenie, skąd pochodzi dana wartość w tabeli wynikowej ani jakie operacje ją wytworzyły. Śledzenie takich zależności jest przedmiotem dziedziny określanej w literaturze jako *data lineage* lub *data provenance* [35, 42]. Brak pełnej informacji o przepływie danych utrudnia audyt i kontrolę jakości danych.

Formalnie problem ten można wyrazić w kategoriach teorii grafów: graf zależności (ang. *lineage graph*) reprezentuje przepływ danych między obiektami bazy takimi jak tabele, widoki, zapytania oraz ich atrybutami. Gdy pewne węzły lub krawędzie zostaną utracone (np. z powodu usunięcia tabeli tymczasowej lub zastosowania funkcji definiowanej przez użytkownika), mówi się o zjawisku *broken lineage* [5].

Celem niniejszej pracy jest *detekcja obiektów bazodanowych o zerwanym lineage* — tabel, które w wyniku broken lineage utraciły producenta lub konsumenta danych. W odróżnieniu od dotychczasowych prac, które modelują broken lineage jako predykcję konkretnych brakujących krawędzi, w pracy sformułowano zadanie na poziomie *węzłów*: uszeregowanie tabel według prawdopodobieństwa, że ich lineage jest niekompletny. Podejście to jest komplementarne wobec predykcji krawędzi i może ją poprzedzać: najpierw wskazuje się podejrzane tabele, a dopiero potem można próbować odtwarzać ich zależności.

W sektorze finansowym banki są zobowiązane do wykazania, które transakcje źródłowe złożyły się na poszczególne pozycje raportów kwartalnych; przerwanie grafu zależności w warstwie ETL (*Extract, Transform, Load*) uniemożliwia przeprowadzenie takiego audytu [5]. Podobne konsekwencje niesie Rozporządzenie o Ochronie Danych Osobowych (RODO): brak pełnego grafu lineage sprawia, że dotarcie do wszystkich kopii danych konkretnej osoby jest czasochłonne lub niemożliwe, co może skutkować nałożeniem kar finansowych [5].

### 1.1 Cel i zakres pracy

Celem pracy jest zbadanie, w jakim stopniu *sama topologia* grafu lineage pozwala wskazać tabele o zerwanej ciągłości pochodzenia danych, bez dostępu do semantyki nazw obiektów, schematów bazy czy ontologii. Zadanie sformułowano jako ranking węzłów typu *Data Table* według prawdopodobieństwa, że tabela została dotknięta zjawiskiem broken lineage; tabele takie nazywane są dalej *zainfekowanymi* (ang. *infected*). Zakres obejmuje:

- metodykę symulacji broken lineage przez usuwanie zadań i wyznaczanie zbioru tabel zainfekowanych jako etykiet referencyjnych,
- porównanie trzech klas metod detekcji: heurystyk strukturalnych, klasycznych klasyfikatorów uczenia maszynowego opartych na cechach pozycji węzła w grafie DAG oraz autorskiego detektora *LineageDetector*,
- ewaluację w protokole *indukcyjnym* cross-graph (trenowanie na części grafów, testowanie na grafach niewidzianych),
- analizę wrażliwości wyniku na odsetek usuwanych zadań oraz kontrolę przecieku wynikającego z trywialnie izolowanych węzłów.

Eksperymenty przeprowadzono na zbiorze danych DLG-DG-23, obejmującym 18 zanonimizowanych grafów lineage ze środowiska Huawei Cloud [10].

Zakres pracy nie obejmuje odtworzenia rzeczywistego schematu bazy danych, analizy treści zanonimizowanych danych ani odtwarzania konkretnych brakujących krawędzi. Predykcja krawędzi została zachowana jako *badanie wstępne*, którego wyniki motywują zmianę ujęcia problemu na poziom węzłów (sekcja 4.1).

## 1.2 Pytania badawcze

Na podstawie systematycznego przeglądu literatury (sekcja 2.8) sformułowano cztery pytania badawcze, na które praca odpowiada w sekcji 5.1:

- PB1** Jakie techniki algorytmiczne są wykorzystywane do ekstrakcji i rekonstrukcji grafów pochodzenia danych ze źródeł nieustrukturyzowanych, takich jak logi zapytań lub częściowe metadane systemowe?
- PB2** W jaki sposób metody uczenia maszynowego mogą zostać zaadaptowane do wykrywania niekompletności grafu zależności bazodanowych?
- PB3** Jakie są ograniczenia obecnych metod w obsłudze struktur niestandardowych — zapytań rekurencyjnych (CTE) oraz funkcji definiowanych przez użytkownika (UDF), stanowiących „czarne skrzynki” przetwarzania?
- PB4** Jakie metryki jakości oraz zbiory danych referencyjnych są właściwe do obiektywnej ewaluacji algorytmów odkrywania lineage?

Pytanie PB2 zostało w toku prac *przeformułowane*. W przeglądzie literatury postawiono je w postaci węższej — jako pytanie o adaptację metod uczenia maszynowego do *predykcji brakujących krawędzi*, zgodnie z dominującym w literaturze ujęciem problemu. Badanie wstępne (sekcja 4.1) wykazało jednak, że przy rzadkości grafów zbioru DLG-DG-23 predykcja krawędzi nie daje wyników odbiegających od poziomu losowego. Pytanie rozszerzono zatem do ogólniejszej postaci przytoczonej wyżej, obejmującej również ujęcie węzłowe — i to właśnie przeformułowanie wyznacza zasadniczy wkład niniejszej pracy.

## 1.3 Wkład własny

Praca wnosi następujące elementy:

1. **Sformułowanie problemu broken lineage na poziomie węzłów.** Zamiast pytać, *która krawędź* została utracona, praca pyta, *które tabele* są dotknięte brakiem, i formalizuje to zadanie jako ranking tabel według prawdopodobieństwa zainfekowania (sekcja 3.1). Ujęcie to nie występuje w dotychczasowej literaturze o lineage bazodanowym (sekcja 2.8.4).
2. **Metodyka symulacji broken lineage przez usuwanie zadań.** Usunięcie całego węzła-zadania wiernie odwzorowuje mechanizm zjawiska i pozwala automatycznie wyznaczyć etykiety referencyjne, bez ręcznego znakowania danych (sekcja 3.3).
3. **Kontrola przecieku izolacji jako element protokołu ewaluacji.** Zidentyfikowano i zmierzono trywialny przeciek „tabela izolowana  $\Rightarrow$  zainfekowana”, obejmujący 19–35% pozytywów, oraz wprowadzono wariant ewaluacji, który go usuwa (sekcja 3.3.1). Element ten jest nieobecny w dotychczasowych pracach o broken lineage.
4. **Autorski detektor LineageDetector.** Łączy cechy pozycji węzła w grafie DAG z relacyjnym score’em kompletności *peer-consistency* i kalibracją prawdopodobieństw (sekcja 3.5.3). Badanie ablacyjne wykazało, że to właśnie sygnał relacyjny odpowiada za przewagę detektora nad klasyfikatorem bazowym (sekcja 4.5).
5. **Ewaluacja o kontrolowanej rzetelności.** Zastosowano protokół indukcyjny cross-graph, testy istotności statystycznej sparowane po grafach, analizę wrażliwości, badanie ablacyjne oraz test na grafie spoza zbioru treningowego. Wyniki raportowane są wraz z przypadkami, w których przewagi wykazać się nie udało (sekcja 4.4).
6. **Odtwarzalna implementacja.** Kod, testy jednostkowe i skrypty odtwarzające każdy eksperyment udostępniono w publicznym repozytorium (sekcja 3.8, dodatek A).

Do wkładu pracy zaliczyć należy również wynik *negatywny*: wykazanie, że na rzeczywistych, rzadkich grafach lineage pozbawionych semantyki nazw predykcja brakujących krawędzi nie przewyższa poziomu losowego (sekcja 4.1). Obserwacja ta jest zbieżna z wynikami uzyskanymi niezależnie metodami grafowo-semantycznymi [15] i stanowi bezpośrednie uzasadnienie zmiany ujęcia problemu.

## 1.4 Struktura pracy

Praca składa się z pięciu rozdziałów. Rozdział 2 zawiera przegląd pojęć podstawowych (data lineage, data provenance, broken lineage oraz detekcja anomalii w grafach), a także omówienie wybranych prac z literatury i stosowanych metryk ewaluacji. Rozdział 3 opisuje problem badawczy, metodykę symulacji broken lineage przez usuwanie zadań, architekturę zaproponowanego rozwiązania oraz zaimplementowane algorytmy detekcji. Rozdział 4 prezentuje wyniki eksperymentów przeprowadzonych na zbiorze DLG-DG-23 wraz z ich analizą. Rozdział 5 zawiera wnioski końcowe, podsumowanie uzyskanych wyników oraz propozycje dalszych kierunków badań.

## Rozdział 2

# Podstawy teoretyczne i przegląd literatury

Niniejszy rozdział wprowadza pojęcia podstawowe niezbędne do zrozumienia dalszych części pracy. W sekcji 2.1 zdefiniowano pojęcia *data lineage* i *data provenance*. Sekcja 2.2 opisuje strukturę grafów *lineage*. Sekcja 2.3 omawia przyczyny niekompletności grafów, a sekcja 2.5 przedstawia zadanie detekcji węzłów o zerwanym *lineage*. Sekcja 2.7 opisuje metryki ewaluacji, a sekcja 2.8 zawiera przegląd wybranej literatury.

### 2.1 Data lineage i data provenance

Pojęcia *data lineage* i *data provenance* są w literaturze stosowane zarówno zamiennie, jak i z różnicowaniem semantycznym [35]. W węższym ujęciu *data lineage* odnosi się do śledzenia przepływu informacji przez kolejne etapy przetwarzania, opisuje, przez jakie transformacje przeszły dane i jak zmieniała się ich postać. Z kolei *data provenance* kładzie nacisk na pochodzenie i autentyczność danych, rejestruje, skąd dane zostały pobrane i kto je wytworzył. W praktyce granica między tymi pojęciami jest płynna i większość opracowań traktuje je jako synonimy. Takie podejście przyjęto również w niniejszej pracy.

Znajomość historii danych ma istotne znaczenie praktyczne. W systemach raportowych możliwość odtworzenia ścieżki transformacji pozwala zlokalizować źródło błędnych wartości, bez tej informacji ręczna weryfikacja wielkich zbiorów danych jest nieefektywna. Innym przykładem jest bezpieczne usuwanie obiektów bazy: tabela pozornie niepowiązana z żadnym procesem może w rzeczywistości dostarczać danych do raportów końcowych, a jej usunięcie spowoduje utratę danych bez widocznego komunikatu błędu [5].

#### 2.1.1 Rodzaje proveniencji

W literaturze wyróżnia się trzy klasyczne rodzaje proveniencji, wprowadzone przez Bunemana et al. [9] i rozwinięte w kolejnych pracach [16, 19]. Proveniencja typu *why* odpowiada na pytanie, które krotki danych źródłowych przyczyniły się do powstania danej krotki wynikowej. Proveniencja *where* wskazuje, z których konkretnie komórek pochodzi wartość. Proveniencja *how* opisuje natomiast *sposób*, w jaki wynik powstał — kombinację operacji, którym poddano dane źródłowe; jej formalizacją są półpierścienie proveniencji [16].

Podział ten odnosi się do poziomu pojedynczych wartości i krotek, określanego jako proveniencja *drobnoziarnista* (ang. *fine-grained*). W praktyce systemów zarządzania danymi stosuje się częściej ujęcie *gruboziarniste* (ang. *coarse-grained*), operujące na poziomie całych obiektów: tabel, widoków i zadań przetwarzania [35]. Powodem jest koszt: śledzenie pochodzenia każdej wartości wymaga instrumentacji silnika bazodanowego i generuje wolumen metadanych porównywalny z sa-



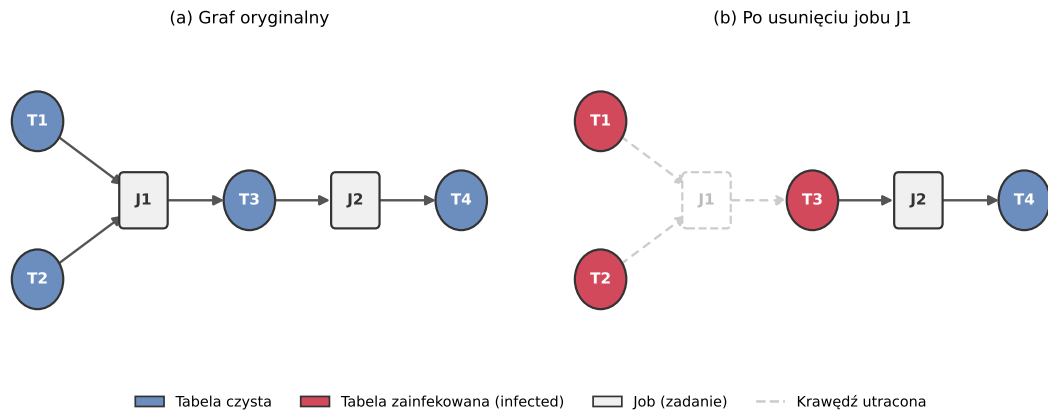
mymi danymi [11]. Niniejsza praca operuje na poziomie gruboziarnistym — przedmiotem analizy są zależności między tabelami a zadaniami, nie zaś pochodzenie poszczególnych wartości.

Standaryzację zapisu proveniencji wprowadza ontologia *PROV-O* [27], rekomendacja W3C definiująca trzy podstawowe klasy — *Entity* (dana), *Activity* (proces) i *Agent* (podmiot odpowiedzialny) — oraz relacje między nimi. Model tabela–zadanie stosowany w niniejszej pracy odpowiada podzbiorowi tej ontologii: tabele pełnią rolę encji, zadania — aktywności, a krawędzie *DATA\_FLOW* odpowiadają relacjom *used* i *wasGeneratedBy*.

## 2.2 Struktura grafu lineage

Graf lineage reprezentuje przepływ danych między obiektami bazy danych. Węzłami są obiekty przetwarzania: tabele i widoki przechowujące dane, zadania (zapytania SQL, procesy ETL, ang. *Extract, Transform, Load*) je przekształcające, oraz atrybuty (pola) tabel. Krawędzie skierowane wyrażają przepływ danych — z tabeli do zadania, które ją czyta, oraz z zadania do tabeli, którą zapisuje — a także relację przynależności pola do tabeli. Podgraf przepływu danych między tabelami a zadaniami jest *dwudzielny* i skierowany, co pozwala interpretować go jako graf skierowany acykliczny (DAG). W niniejszej pracy przedmiotem detekcji są wyłącznie węzły reprezentujące tabele.

Rysunek 2.1 przedstawia poglądowy fragment grafu przepływu danych przed usunięciem zadania i po jego usunięciu. Usunięcie zadania J1 pozbawia tabele T1 i T2 konsumenta, a tabelę T3 — producenta; wszystkie trzy stają się zainfekowane, choć pozostają w grafie (T3 zachowuje połączenie z zadaniem J2).



RYSUNEK 2.1: Fragment grafu przepływu danych: (a) graf oryginalny, (b) graf po usunięciu zadania J1. Tabele zainfekowane oznaczono kolorem czerwonym, krawędzie utracone — linią przerywaną.

Wszystkie eksperymenty oparto na zbiorze danych DLG-DG-23; jego pełną charakterystykę — konkretną reprezentację typów węzłów i krawędzi, anonimizację oraz statystyki 18 grafów — zebrano w sekcji 3.2.

## 2.3 Niekompletność grafów lineage — broken lineage

Graf lineage bywa niekompletny z kilku powodów. Tabele tymczasowe, tworzone na potrzeby pojedynczej sesji lub procesu ETL, znikają po jego zakończeniu. Jeśli pośredniczyły w przepływie

danych między dwoma trwałymi tabelami, znika wraz z nimi odpowiedni fragment grafu. Analogiczny skutek mają widoki zmaterializowane tworzone tymczasowo oraz funkcje definiowane przez użytkownika (ang. *User-Defined Function*, UDF), które opakowują logikę transformacji bez jawnego zapisu przepływu danych [42].

Zjawisko to, określane jako *broken lineage*, sprawia, że tabela traci część lub całość informacji o swoim pochodzeniu: przestaje mieć widocznego producenta danych (zadanie zapisujące) lub ich konsumenta (zadanie czytające), mimo że rzeczywista zależność istnieje. W niniejszej pracy problem *broken lineage* jest modelowany jako zadanie *detekcji zainfekowanych węzłów* — dla każdej tabeli algorytm ocenia prawdopodobieństwo, że jej lineage został zerwany. Formalne ujęcie problemu przedstawiono w rozdziale 3.

### 2.3.1 Taksonomia przyczyn

Tabela 2.1 systematyzuje mechanizmy prowadzące do niekompletności grafu lineage. Wspólny im jest schemat: transformacja zachodzi, lecz pozostaje niewidoczna dla narzędzi zbierających metadane, przez co w grafie brakuje węzła pośredniczącego albo krawędzi.

TABELA 2.1: Przyczyny *broken lineage* i ich konsekwencje dla grafu zależności.

| Przyczyna                                   | Mechanizm                                                                                     | Skutek w grafie                                                       |
|---------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Tabela tymczasowa                           | obiekt pośredniczący usuwany po zakończeniu sesji lub procesu ETL                             | znika węzeł pośredni; tabela źródłowa i docelowa stają się rozłączone |
| Funkcja definiowana przez użytkownika (UDF) | logika transformacji ukryta w kodzie proceduralnym, nieanalizowanym przez parser SQL [42]     | krawędź istnieje w rzeczywistości, lecz nie jest rejestrowana         |
| Zapytanie rekurencyjne (CTE)                | wielokrotne odwołanie do tej samej relacji utrudnia jednoznaczne przypisanie pochodzenia [14] | zależność zapisana niepełnie lub pominięta                            |
| Widok zmaterializowany                      | tworzony tymczasowo na potrzeby jednego procesu                                               | jak przy tabeli tymczasowej                                           |
| Przetwarzanie zewnętrzne                    | transformacja wykonana poza silnikiem bazy (skrypt, zadanie Spark)                            | brak jakiegokolwiek śladu w metadanych bazy                           |
| Partycjonowanie i kopiowanie                | dane rozdzielane lub replikowane bez jawnego zapisu relacji                                   | rozerwanie pojedynczej zależności na wiele nieudokumentowanych        |

W niniejszej pracy wszystkie te mechanizmy modelowane są jednolicie — jako usunięcie węzła zadania wraz z jego krawędziami (sekcja 3.3). Uproszczenie to jest celowe: detekcja opiera się na *skutku* zerwania widocznym w topologii, a nie na jego przyczynie, dzięki czemu metoda nie wymaga osobnej obsługi każdego mechanizmu.

## 2.4 Narzędzia zarządzania lineage w praktyce

Zagadnienie omawiane w pracy ma bezpośrednie odzwierciedlenie w narzędziach przemysłowych klasy *data governance*. Tabela 2.2 zestawia najczęściej stosowane rozwiązania wraz ze sposobem pozyskiwania przez nie informacji o pochodzeniu danych.

Wspólne ograniczenie tych rozwiązań jest istotne dla uzasadnienia niniejszej pracy: wszystkie zbierają lineage *prospektywnie* — w chwili wykonania zadania i tylko dla zadań objętych instrumentacją. Nie oferują natomiast mechanizmu *retrospektywnego* wskazania miejsc, w których zebrany graf jest niekompletny. Tam, gdzie instrumentacja zawiodła albo nie istniała (systemy zastane, przetwarzanie poza silnikiem bazy, audyt post hoc), narzędzie nie sygnalizuje braku — po prostu

TABELA 2.2: Wybrane narzędzia zarządzania lineage i stosowane przez nie metody akwizycji metadanych.

| Narzędzie                  | Metoda akwizycji                                        | Podatność na broken lineage                                                         |
|----------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------|
| Apache Atlas [3]           | wtyczki ( <i>hooks</i> ) do silników Hive, Spark, Kafka | brak śladu przy przetwarzaniu poza zintegrowanym silnikiem                          |
| OpenLineage / Marquez [33] | otwarty standard zdarzeń emitowanych przez zadania      | wymaga instrumentacji każdego zadania; zadania nieobsługiwane pozostają niewidoczne |
| Spline                     | przechwytywanie planów wykonania Apache Spark           | ograniczone do jednego silnika przetwarzania                                        |
| DataHub, Collibra          | parsowanie SQL oraz import metadanych z katalogów       | parser nie sięga do wnętrza UDF ani kodu proceduralnego                             |

pokazuje graf uboższy od rzeczywistego. Detekcja tabel o zerwanym lineage, będąca przedmiotem pracy, adresuje właśnie tę lukę.

## 2.5 Detekcja węzłów o zerwanym lineage

Detekcja anomalii na poziomie węzłów grafu polega na przypisaniu każdemu węzłowi wyniku (score) wyrażającego stopień, w jakim odbiega on od wzorca uznawanego za poprawny, a następnie na uszeregowaniu węzłów według tego wyniku. W kontekście grafów lineage węzłem podejrzanym jest tabela, której sąsiedztwo w grafie DAG sugeruje niekompletność pochodzenia danych.

Dziedzina detekcji anomalii w grafach dysponuje ustaloną taksonomią metod [1, 30]. Metody *oparte na cechach* opisują węzeł wektorem statystyk strukturalnych (stopień, współczynnik growowania, miary centralności), a następnie stosują klasyfikator lub detektor wartości odstających. Metody *spektralne* i oparte na faktoryzacji wykrywają węzły odbiegające od struktury niskiego rzędu przybliżającej graf. Metody *oparte na głębokim uczeniu* — w szczególności grafowe sieci neuronowe — uczą reprezentacji węzła z propagacją komunikatów między sąsiadami. Niniejsza praca sytuuje się w pierwszej z tych rodzin: węzeł opisywany jest wektorem cech pozycji w grafie DAG, uzupełnionym o sygnał relacyjny, a decyzję podejmuje klasyfikator uczony indukcyjnie. Wybór ten podyktowany jest rozmiarem zbioru — 18 grafów to zbyt mało, by trenować model głęboki bez ryzyka przeuczenia — oraz celem pracy, którym jest wyznaczenie punktu odniesienia dla metod bogatszych, a nie maksymalizacja skuteczności.

Zasadnicza różnica względem klasycznej detekcji anomalii polega tu na tym, że anomalia nie jest własnością wewnętrzną węzła, lecz skutkiem *usunięcia* innego węzła. Tabela zainfekowana nie wygląda dziwnie sama w sobie — wygląda jak poprawna tabela innego rodzaju (źródłowa lub końcowa). Odróżnienie wymaga zatem porównania z kontekstem, nie zaś wykrycia wartości odstających.

W przeciwieństwie do predykcji krawędzi, zadanie nie polega na wskazaniu *której* krawędzi brakuje, lecz *które* tabele są dotknięte brakiem. Jest to zadanie rankingowe, oceniane metrykami czuły na uszeregowanie (sekcja 2.7), a nie zadanie klasyfikacji przy ustalonym progu. Motywacja dla tego ujęcia wynika z ograniczeń danych DLG-DG-23: rzadkość grafów sprawia, że sygnał o konkretnej brakującej parze obiektów jest zbyt słaby (sekcja 4.1), natomiast wskazanie podejrzanych tabel pozostaje wykonalne.

W pracy zastosowano trzy klasy metod detekcji:

1. **Heurystyki strukturalne** — reguły oparte wyłącznie na pozycji węzła w grafie (stopień, rola korzeń/liść) oraz autorski score kompletności *peer-consistency*; nie wymagają uczenia.

2. **Klasyczne metody ML** — Random Forest, RUSBoost i LightGBM; klasyfikują węzeł na podstawie wektora cech pozycji w DAG. Ze względu na anonimizację (UUID) metryki podobieństwa nazw (np. Jaro-Winkler stosowane w [5]) są nieaplikowalne.
3. **Autorski detektor LineageDetector** — łączy cechy pozycji w DAG, relacyjny score kompletności oraz kalibrację prawdopodobieństw (isotonic); jego wynikiem jest oszacowanie  $p(\text{zainfekowana})$ .

## 2.6 Wykorzystane metody uczenia maszynowego

Sekcja przedstawia podstawy teoretyczne metod zastosowanych w rozdziale 3. Ich wspólnym mianownikiem jest zdolność do pracy przy silnie niezrównoważonym rozkładzie klas, charakterystycznym dla zadania detekcji.

### 2.6.1 Metody zespołowe

**Random Forest** [6] buduje zespół drzew decyzyjnych, z których każde uczy się na losowej próbce obserwacji (*bagging*) i rozważa w każdym podziale losowy podzbiór cech. Uśrednienie predykcji redukuje wariancję pojedynczego drzewa, zachowując jego zdolność do modelowania nieliniowych interakcji między cechami. Niezrównoważenie klas kompensuje się wagami odwrotnie proporcjonalnymi do ich liczności.

**Boosting gradientowy** buduje drzewa sekwencyjnie: każde kolejne minimalizuje błąd reszkowy poprzedników. *LightGBM* [22] wprowadza dwa usprawnienia obniżające koszt obliczeniowy — próbkowanie obserwacji o dużym gradiencie oraz łączenie cech wzajemnie wykluczających się — oraz rozrost drzewa liśćmi zamiast poziomami, co przy tej samej liczbie węzłów daje mniejszy błąd.

**RUSBoost** [38] łączy boosting typu AdaBoost z losowym podpróbkowaniem klasy większościowej przed każdą iteracją. Metoda powstała specjalnie dla danych niezrównoważonych i jest tańsza obliczeniowo od podejść opartych na generowaniu obserwacji syntetycznych. W niniejszej pracy zastosowano ją przede wszystkim dla bezpośredniej porównywalności z wynikami Boińskiego et al. [5], u których osiągnęła najlepszy rezultat.

### 2.6.2 Kalibracja prawdopodobieństw

Klasyfikator zwracający poprawny *ranking* nie musi zwracać poprawnych *prawdopodobieństw*. Metody zespołowe oparte na drzewach są pod tym względem systematycznie obciążone: przesuwają oszacowania ku środkowi przedziału, rzadko wskazując wartości bliskie 0 lub 1 [32]. Ponieważ zadanie postawione w niniejszej pracy wymaga oszacowania  $p(\text{tabela zainfekowana})$ , a nie tylko uszeregowania, zastosowano kalibrację.

*Regresja izotoniczna* [43] dopasowuje niemalejącą funkcję schodkową przekształcającą surowy wynik klasyfikatora w prawdopodobieństwo, minimalizując błąd kwadratowy na zbiorze kalibracyjnym. W odróżnieniu od kalibracji Platt'a nie zakłada określonej postaci funkcyjnej, jest jednak bardziej podatna na przeuczenie przy małej liczbie obserwacji. Jej istotną własnością — mającą znaczenie dla wyników rozdziału 4 — jest schodkowy charakter: obserwacjom o zbliżonych wynikach przypisywane jest identyczne prawdopodobieństwo, co wprowadza do rankingu remisę nieobecne w wyniku surowym.

## 2.7 Metryki ewaluacji

Zadanie detekcji ma charakter rankingowy, dlatego stosuje się metryki oceniające jakość uszeregowania węzłów, a nie pojedynczej decyzji przy progu:

- **AUC-ROC** — pole pod krzywą ROC; mierzy zdolność modelu do postawienia zainfekowanej tabeli wyżej w rankingu niż czystej,
- **AUC-PR** — pole pod krzywą precyzja–czułość; szczególnie istotne przy niezrównoważonych klasach, gdy liczba tabel czystych wielokrotnie przewyższa liczbę zainfekowanych,
- **Precision@k** — frakcja tabel faktycznie zainfekowanych wśród  $k$  najwyżej sklasyfikowanych,
- **Recall@k** — frakcja wszystkich zainfekowanych tabel odzyskanych w pierwszych  $k$  pozycjach rankingu,
- **Hits@k** — czy wśród  $k$  czołowych pozycji znajduje się co najmniej jedna tabela zainfekowana,
- **Brier score** — jakość kalibracji prawdopodobieństw (istotna dla detektora zwracającego  $p(\text{zainfekowana})$ ).

Metryki rankingowe są szczególnie istotne w warunkach niezrównoważonych klas, charakterystycznych dla grafów rzadkich, gdzie liczba tabel czystych wielokrotnie przewyższa liczbę tabel zainfekowanych [5, 25]. Spośród wymienionych miar AUC-ROC jest w takich warunkach najbardziej optymistyczna: pozostaje niewrażliwa na rozkład klas, przez co może sugerować dobrą skuteczność również wtedy, gdy bezwzględna liczba trafień w czołówce rankingu jest niewielka. Dlatego jako miarę uzupełniającą raportuje się AUC-PR, która wprost odzwierciedla udział klasy pozytywnej i w zadaniach niezrównoważonych rozdziela metody wyraźniej niż AUC-ROC [12, 36]. Brier score [7], zdefiniowany jako średni kwadrat różnicy między oszacowanym prawdopodobieństwem a etykietą, jest natomiast jedyną ze stosowanych metryk oceniającą nie kolejność, lecz rzetelność samych oszacowań.

## 2.8 Systematyczny przegląd literatury

Stan wiedzy rozpoznano systematycznym przeglądem literatury (ang. *Systematic Literature Review*, SLR), przeprowadzonym zgodnie z wytycznymi Kitchenham i Chartersa [24]. Sekcja 2.8.1 opisuje protokół przeglądu i przebieg selekcji, sekcje kolejne — wyniki: omówienie prac wybranych do analizy pełnotekstowej oraz wynikającą z nich lukę badawczą.

Wyprzedzając szczegóły: wszystkie zidentyfikowane prace dotyczące broken lineage między obiektami bazy modelują problem jako *predykcję krawędzi*; detekcji na poziomie węzłów — wskazania, które tabele mają zerwany lineage — literatura ta nie podejmuje. Tę lukę wypełnia niniejsza praca. Omówione dalej prace pogrupowano w dwie kategorie: prace *bazowe*, stanowiące bezpośredni fundament niniejszej pracy (metoda kontrastowa oraz wykorzystany zbiór danych referencyjnych), oraz *metody pokrewne* odkrywania zależności lineage, wyznaczające szerszy kontekst metodyczny.

### 2.8.1 Metodyka przeglądu

#### Strategia wyszukiwania

Wyszukiwanie przeprowadzono w bazie *DBLP Computer Science Bibliography* — uznanym źródle indeksującym recenzowane publikacje informatyczne, obejmującym zarówno czasopisma,

jak i materiały konferencyjne. Ze względu na specyfikę tematu zapytanie ograniczono do dwóch terminów kluczowych, połączonych alternatywą, co zapewnia szerokie pokrycie dziedziny przy zachowaniu precyzji:

"data provenance"OR "data lineage"

Terminy te dobrano celowo szeroko. Węższe zapytanie, zawierające wprost frazę *broken lineage* lub *missing lineage*, pomijałoby prace opisujące to samo zjawisko innym słownictwem (np. *lineage reconstruction*, *provenance completion*); przesunięto zatem ciężar selekcji z zapytania na etap przesiewania.

### Kryteria selekcji

**Zakres czasowy.** Analizie poddano publikacje z lat 2022–2025. Ograniczenie to wynika z tempa rozwoju metod uczenia maszynowego w inżynierii danych: prace wcześniejsze opisują w większości podejścia oparte na statycznej analizie schematów, nieprzystające do współczesnych architektur przetwarzania.

**Kryteria włączenia.** Publikacje w języku angielskim, proponujące konkretny algorytm lub architekturę możliwą do zaimplementowania i zweryfikowania eksperymentalnie.

**Kryteria wyłączenia.** Prace spoza domeny inżynierii oprogramowania i baz danych (termin *lineage* występuje licznie m.in. w biologii — *cell lineage tracing*); publikacje dotyczące wyłącznie wizualizacji grafów pochodzenia, bez wkładu algorytmicznego; prace skupione wyłącznie na aspektach bezpieczeństwa (szyfrowanie, blockchain), bez analizy struktury zależności między danymi.

### Przebieg selekcji

Selekcję przeprowadzono dwuetapowo — najpierw na podstawie tytułów, następnie abstraktów — a na końcu wykonano analizę pełnotekstową prac, które przeszły oba etapy. Tabela 2.3 przedstawia liczby publikacji na poszczególnych etapach.

TABELA 2.3: Przebieg selekcji literatury w bazie DBLP dla zapytania "data provenance"OR "data lineage".

| Etap           | Zastosowane działanie                 | Publikacji |
|----------------|---------------------------------------|------------|
| Wyniki surowe  | wyszukiwanie bez ograniczeń czasowych | 1256       |
| Filtrowanie    | lata 2022–2025, język angielski       | 288        |
| Przesiewanie 1 | selekcja na podstawie tytułów         | 24         |
| Przesiewanie 2 | selekcja na podstawie abstraktów      | 17         |
| Wybór końcowy  | analiza pełnotekstowa                 | 7          |

Największy ubytek następuje na etapie przesiewania tytułów ( $288 \rightarrow 24$ ) i wynika z wieloznaczności terminu *lineage* poza informatyką. Drugie przesiewanie ( $24 \rightarrow 17$ ) usunęło prace czysto teoretyczne oraz pozbawione opisu technicznego pozwalającego na odtworzenie metody. Analiza pełnotekstowa zawężyła zbiór do siedmiu prac reprezentujących odmienne podejścia algorytmiczne: klasyczne uczenie maszynowe, analizę logów zapytań, grafowe sieci neuronowe, metody jądrowe oraz przetwarzanie zapytań rekurencyjnych i funkcji definiowanych przez użytkownika.

### Uzupełnienie przeglądu o prace spoza bazy

Systematyczny przegląd przeprowadzono na początku prac nad pracą magisterską. W jego trakcie nie zostały uwzględnione trzy publikacje, które omówiono w dalszej części rozdziału: Dutkiewicz et al. [15], Andrzejewski et al. [2] oraz Brzeski i Roman [8]. Pochodzą one z lat 2025–2026

i w chwili wykonywania zapytania nie były jeszcze zaindeksowane w bazie DBLP; dotarto do nich później, w drodze wyszukiwania ukierunkowanego oraz konsultacji z zespołem badawczym promotora. Uzupełnienie zbioru prac o pozycje wskazane przez ekspertów dziedzinowych jest praktyką dopuszczalną w przeglądach systematycznych, wymaga jednak jawnego zadeklarowania — prace te nie przeszły bowiem opisanej wyżej procedury selekcji. Ich rola w niniejszej pracy jest przy tym istotna: wyznaczają one bezpośredni kontekst metodyczny, w którym plasuje się postawione pytanie badawcze.

### 2.8.2 Prace bazowe: metoda kontrastowa i zbiór danych

#### Leveraging machine learning techniques for discovering broken lineage links

**Autorzy:** Boiński, Andrzejewski, Grocholewski, Gruszczyński, Wrembel [5].

**Problem badawczy.** Ciągłość grafów *data lineage* ulega przerwaniu, gdy w potokach ETL wykorzystywane są obiekty pośrednie — tabele tymczasowe lub UDF. W rezultacie narzędzia klasy *data governance* tracą ślad przepływu danych i błędnie interpretują trwałą tabelę źródłową oraz tabelę docelową jako byty niezależne.

**Zaproponowane rozwiązanie.** Autorzy formułują problem rekonstrukcji przerwanych zależności jako binarną klasyfikację par obiektów bazodanowych, z inżynierią cech opartą na metadanych i podobieństwie nazw (Jaro–Winkler), klasyfikatorami Random Forest, RUSBoost, BalancedBagging i LightGBM oraz generowaniem etykiet metodą rekurencyjnego zastępowania tabel tymczasowych ich trwałymi źródłami. Ze względu na brak publicznych danych autorzy wygenerowali syntetyczne schematy baz LLM-ami.

**Analiza krytyczna i relacja do pracy.** Najwyższą skuteczność osiągnął RUSBoost ( $F1 = 0,79$ ). Artykuł ten jest punktem wyjścia niniejszej pracy, która jednak *istotnie się od niego różni*: Boiński et al. przewidują *krawędzie* i korzystają z semantyki nazw, podczas gdy w niniejszej pracy detekcja odbywa się na poziomie *węzłów* i wyłącznie na podstawie topologii (dane DLG-DG-23 są zanonimizowane). Adaptowano natomiast rodzinę klasyfikatorów (RF, RUSBoost, LightGBM) jako punkty odniesienia dla autorskiego detektora.

#### An open dataset of data lineage graphs

**Autorzy:** Chen, Zhao, Li, Zhang, Long, Zhou [10].

**Problem badawczy.** Rozwój metod w obszarze *data governance* jest ograniczany przez brak ogólnodostępnych, rzeczywistych zbiorów danych; przedsiębiorstwa nie udostępniają grafów lineage ze względu na poufność.

**Zaproponowane rozwiązanie.** Autorzy opublikowali zbiór DLG-DG-23 — pierwszy otwarty zestaw rzeczywistych grafów pochodzenia danych: 18 grafów w zakresie 278–17 085 węzłów, ze środowiska produkcyjnego Huawei Cloud (dane zanonimizowane), z węzłami trzech typów (tabela, zadanie, pole) i relacjami przepływu oraz hierarchii.

**Analiza krytyczna i relacja do pracy.** Wartość zbioru wynika z jego rzeczywistego, przemysłowego pochodzenia; ograniczeniem jest niewielka liczba grafów oraz anonimizacja atrybutów. W niniejszej pracy DLG-DG-23 jest referencyjnym zbiorem danych (*ground truth*) we wszystkich eksperymentach.

### 2.8.3 Metody pokrewne odkrywania zależności lineage

#### Data lineage discovery in databases based on knowledge graph link prediction

**Autorzy:** Dutkiewicz, Misiorek, Wrembel [15].

**Problem badawczy.** Automatyczne odkrywanie brakujących zależności lineage między obiektami bazy danych jest słabo zbadane, ponieważ rzeczywiste dane biznesowe i schematy baz nie są publicznie dostępne — w konsekwencji brakuje benchmarków umożliwiających rzetelną ewaluację metod odkrywania.

**Zaproponowane rozwiązanie.** Praca wnosi trzy artefakty: (1) procedurę generowania scenariuszy broken lineage za pomocą frameworku opartego na LLM, obejmującą dziewięć typów scenariuszy (m.in. kopiowanie danych, transformacje, złączenia, deduplikację, widoki, zapytania rekurencyjne, partycjonowanie i tabele tymczasowe); (2) model semantyczny transformujący scenariusze do grafu wiedzy (RDF, relacja *valueDerivedFrom* inspirowana ontologią PROV-O); (3) dostosowany do lineage protokół ewaluacji *w pełni indukcyjnej* predykcji krawędzi ( $\mathcal{E}_{\text{train}} \cap \mathcal{E}_{\text{test}} = \emptyset$ ) z użyciem path-based GNN, próbkowaniem negatywów 50:1 oraz metrykami AUROC i Hits@10.

**Analiza krytyczna i relacja do pracy.** Wyniki są silnie zależne od typu scenariusza: proste kopiowanie danych osiąga AUROC 0,92 (model dedykowany), lecz scenariusze złożone spadają do  $\approx 0,60$ , a *tabele tymczasowe* — najczęstsza praktyczna przyczyna broken lineage — do AUROC 0,49, czyli poziomu losowego. Potwierdza to trudność problemu nawet dla zaawansowanych metod grafowo-semantycznych. Z pracy tej zaadaptowano do niniejszej pracy *protokół indukcyjny cross-graph* oraz zestaw metryk rankingowych. Zasadnicza różnica: Dutkiewicz et al. przewidują krawędzie w grafie wiedzy wzbogaconym semantycznie, podczas gdy niniejsza praca operuje na poziomie węzłów i czystej topologii.

#### On discovering provenance of data warehouse objects created by select-project queries

**Autorzy:** Andrzejewski, Boiński, Wrembel [2].

**Problem badawczy.** W hurtowniach danych obiekty (np. widoki zmaterializowane) powstają z innych obiektów przez zapytania select-project, a potoki integracji danych tworzą obiekty tymczasowe i ulotne (tabele tymczasowe, zmienne globalne, kursory), których zależności nie są automatycznie rejestrowane i zanikają po zakończeniu zadania.

**Zaproponowane rozwiązanie.** Podejście analityczne (niewymagające uczenia maszynowego): (1) algorytm odzyskujący współczynniki (nachylenie i wyraz wolny) funkcji liniowej zastosowanej w projekcji oraz (2) algorytm odtwarzający warunek selekcji użyty w zapytaniu. Zależność między obiektami jest wykrywana przez dopasowanie modelu matematycznego do *wartości* przechowywanych danych.

**Analiza krytyczna i relacja do pracy.** Metoda daje dokładne, interpretowalne wyniki, lecz jej założenia są restrykcyjne: wymaga dostępu do zawartości tabel (wartości danych) oraz ogranicza się do transformacji liniowych w zapytaniach select-project. W warunkach niniejszej pracy (zanonimizowane grafy bez dostępu do danych i schematów) podejście to jest nieaplikowalne. Ograniczenie to uzasadnia pytanie postawione w niniejszej pracy: jak daleko sięga sygnał czysto topologiczny.

#### Inferring missing data lineage links from schema metadata using transformer-based models

**Autorzy:** Brzeski, Roman [8].



**Problem badawczy.** Istniejące metody ekstrakcji lineage (instrumentacja, analiza czasu wykonania, adnotowane potoki) zawodzą, gdy informacja o przepływie jest częściowo niedostępna — np. w środowiskach hybrydowych, systemach zastanych (*legacy*) lub przy audytach post hoc.

**Zaproponowane rozwiązanie.** Dwustopniowa metoda wnioskowania *kierunkowych* zależności lineage na poziomie kolumn wyłącznie z metadanych schematu, inspirowana technikami dopasowywania schematów: bi-encoder transformerowy filtruje kandydatów, a cross-encoder wykonuje precyzyjną predykcję. Ewaluacja na wielkoskalowych danych komercyjnych z ekstremalną nierównowagą klas (1 poprawny link na 90 000 par kandydackich).

**Analiza krytyczna i relacja do pracy.** Praca pokazuje, że przy dostępnej semantyce nazw kolumn i schematów modele transformerowe skutecznie odzyskują zależności mimo skrajnej nierównowagi klas. Warunkiem jest jednak dokładnie ta informacja, której w zbiorze DLG-DG-23 brak (identyfikatory UUID) — sygnał semantyczny i sygnał topologiczny badany w niniejszej pracy są zatem komplementarne, a ich połączenie jest naturalnym kierunkiem dalszych badań.

#### 2.8.4 Synteza porównawcza

Tabela 2.4 zestawia omówione prace według czterech wymiarów rozstrzygających o ich relacji do niniejszej pracy: poziomu, na którym formułowany jest problem, źródła wykorzystywanego sygnału, rodzaju danych ewaluacyjnych oraz zastosowanego protokołu oceny.

TABELA 2.4: Zestawienie porównawcze omówionych prac. Ostatni wiersz — niniejsza praca.

| Praca                      | Poziom              | Źródło sygnału        | Dane              | Protokół     |
|----------------------------|---------------------|-----------------------|-------------------|--------------|
| Boiński et al. [5]         | krawędź (para)      | nazwy + metadane      | syntetyczne (LLM) | klasyfikacja |
| Chen et al. [10]           | — (zbiór danych)    | —                     | rzeczywiste       | —            |
| Dutkiewicz et al. [15]     | krawędź             | graf wiedzy (RDF)     | syntetyczne       | indukcyjny   |
| Andrzejewski et al. [2]    | parametry zapytania | wartości danych       | —                 | analityczny  |
| Brzeski, Roman [8]         | krawędź (kolumny)   | metadane schematu     | komercyjne        | klasyfikacja |
| Psallidas et al. [35]      | ekstrakcja          | logi zapytań          | rzeczywiste       | —            |
| Yamada et al. [42]         | ekstrakcja (UDF)    | analiza kodu          | rzeczywiste       | —            |
| Dietrich et al. [14]       | ekstrakcja (CTE)    | przepisywanie zapytań | —                 | —            |
| Kohan Marzagão et al. [25] | podobieństwo grafów | topologia             | rzeczywiste       | —            |
| Shan et al. [39]           | węzeł               | heterogeniczny GNN    | systemowe         | —            |
| Niniejsza praca            | węzeł (tabela)      | wyłącznie topologia   | rzeczywiste       | indukcyjny   |

Zestawienie uwidacznia dwie rzeczy. Po pierwsze, kolumna „poziom” jest niemal w całości wypełniona wartością *krawędź* lub *ekstrakcja* — jedynym podejściem węzłowym jest praca Shana et al. [39], dotycząca jednak systemowych grafów provenance w kontekście bezpieczeństwa, nie zaś obiektów bazodanowych. Po drugie, żadna z prac operujących na lineage bazodanowym nie ogranicza się do sygnału czysto topologicznego: wszystkie korzystają z nazw, metadanych schematu, wartości danych lub semantyki grafu wiedzy. Skrzyżowanie tych dwóch wymiarów — poziom węzła i wyłącznie topologia — pozostaje niezajęte.

#### 2.8.5 Luka badawcza

Wszystkie omówione prace — od klasycznego ML na cechach semantyczno-topologicznych [5], przez metody analityczne [2] i transformery na metadanych schematu [8], po indukcyjne GNN na grafach wiedzy [15] — modelują broken lineage jako odtwarzanie *zależności* (krawędzi lub parametrów transformacji). Żadna nie stawia pytania, *które tabele mają zerwany lineage*. Niniejsza praca uzupełnia ten obraz: wskazanie podejrzanych tabel może poprzedzać kosztowniejsze odtwarzanie konkretnych zależności.

### 2.8.6 Prace kontekstowe

Spośród pozostałych prac na uwagę zasługuje metoda *Provenance Graph Kernel* (PGK) [25], definiująca miary podobieństwa grafów proveniencji. Detekcja anomalii na poziomie węzłów pojawia się w pokrewnej dziedzinie systemowych grafów provenance i bezpieczeństwa (np. heterogeniczne GNN [39]), nie była jednak dotąd stosowana do lineage obiektów bazodanowych. Prace dotyczące proveniencji rekurencyjnych zapytań SQL [14], analizy logów [35] oraz UDF [42] tworzą szerszy kontekst literaturowy.

## Rozdział 3

# Problem i sposób jego rozwiązania

Rozdział precyzuje formalną definicję problemu detekcji węzłów o zerwanym lineage (sekcja 3.1). Sekcja 3.2 charakteryzuje zastosowany zbiór danych, sekcja 3.3 — metodykę symulacji broken lineage przez usuwanie zadań, a sekcja 3.4 opisuje architekturę potoku przetwarzania. Szczegóły implementacji metod detekcji omówiono w sekcji 3.5.

### 3.1 Sformułowanie problemu

Graf lineage modeluje się jako skierowany graf DAG przepływu danych  $G = (V, E)$ , gdzie  $V = V_J \cup V_T \cup V_F$  obejmuje węzły typu zadanie (*Data Job*), tabela (*Data Table*) i pole, a  $E$  zawiera krawędzie przepływu danych (*DATA\_FLOW*) między zadaniami a tabelami. Zjawisko broken lineage odpowiada usunięciu z grafu pewnego zadania  $J$ : tabele z nim incydentne — jego poprzedniki (tabele karmiące  $J$ ) oraz następniki (tabele karmione przez  $J$ ) — tracą część informacji o swoim pochodzeniu i stają się *zainfekowane* (ang. *infected*).

Obserwowany graf ma postać  $G_{\text{obs}} = G \setminus \{J_1, \dots, J_m\}$ , powstałą przez usunięcie podzbioru zadań. Zadaniem detekcji jest wyznaczenie funkcji, która każdej tabeli  $t \in V_T$  przypisuje wynik  $s(t) \in [0, 1]$  interpretowany jako oszacowane prawdopodobieństwo  $p$ (tabela  $t$  jest zainfekowana). Wynikiem nie jest decyzja binarna przy ustalonym progu, lecz *ranking* tabel — tabele o najwyższym wyniku są pierwszymi kandydatami do ręcznej inspekcji.

Etykiety referencyjne (*ground truth*) generowane są automatycznie: tabela jest zainfekowana wtedy i tylko wtedy, gdy była incydentna do co najmniej jednego usuniętego zadania. Zbiór kandydatów ograniczono do węzłów typu *Data Table* — węzły typu pole (*field*) łączy z tabelami wyłącznie relacja *PARENT\_CHILD* i nie podlegają one detekcji.

Zasadniczą trudność zadania polega na tym, że osierocona tabela-następnik po usunięciu zadania wygląda jak prawdziwa tabela źródłowa ( $\text{in\_df} = 0$ ). Rozróżnienie tych dwóch przypadków wymaga cech opisujących pozycję węzła w grafie DAG oraz uczenia *indukcyjnego* (cross-graph), a nie samego stopnia węzła.

### 3.2 Zbiór danych DLG-DG-23

Zastosowany zbiór danych pochodzi z pracy Chen et al. [10] i zawiera 18 grafów lineage pobranych ze środowiska produkcyjnego Huawei Cloud. Dane zostały zanonimizowane — nazwy tabel i atrybutów zastąpiono identyfikatorami w formacie UUID, a schematy bazy danych nie zostały udostępnione. Zbiór opublikowano w 2024 roku jako pierwszy otwarty zestaw rzeczywistych grafów lineage.

Każdy graf zawiera węzły trzech typów — **Data Job** (zapytanie SQL lub zadanie ETL, ang. *Extract, Transform, Load*), **Data Table** (tabela lub widok przechowujący dane) oraz **field** (atrybut/kolumna należąca do tabeli) — połączone krawędziami dwóch typów: **DATA\_FLOW** (skierowana krawędź przepływu danych między tabelą a zadaniem) oraz **PARENT\_CHILD** (relacja przynależności pola do tabeli). Pełny graf jest zatem *heterogeniczny*. Przedmiotem detekcji są wyłącznie węzły typu *Data Table*, a sygnał czerpany jest z podgrafu przepływu danych (krawędzie *DATA\_FLOW* między węzłami *Data Table* i *Data Job*); podgraf ten ma strukturę dwudzielną i skierowaną, interpretowaną jako graf DAG. Grafy różnią się skalą: liczba węzłów waha się od 278 do 17 085.

Zbiór DLG-DG-23 wybrano, ponieważ jest jedynym publicznie dostępnym zestawem rzeczywistych grafów lineage. Anonimizacja nazw węzłów ma bezpośredni wpływ na dobór metod: metryki podobieństwa oparte na nazwach obiektów (np. Jaro–Winkler stosowana przez Boińskiego et al. [5]) są nieaplikowalne. W niniejszej pracy sygnał ograniczono zatem do atrybutów *topologicznych* grafu. Ograniczenie to jest zarazem częścią pytania badawczego: czy sama struktura grafu lineage wystarcza do wskazania tabel o zerwanym pochodzeniu danych. Metoda skuteczna przy tak ograniczonym wglądzie w graf ma przy tym większą szansę na zastosowanie praktyczne niż podejścia wymagające pełnych metadanych.

### 3.2.1 Charakterystyka grafów

Tabela 3.1 przedstawia statystyki poszczególnych grafów zbioru DLG-DG-23.

TABELA 3.1: Charakterystyka grafów zbioru DLG-DG-23.

| ID    | Węzły  | Krawędzie DATA_FLOW | Skala  |
|-------|--------|---------------------|--------|
| DLG1  | 298    | 24                  | mały   |
| DLG2  | 464    | 37                  | mały   |
| DLG3  | 603    | 57                  | mały   |
| DLG4  | 415    | 32                  | mały   |
| DLG5  | 1 299  | 175                 | średni |
| DLG6  | 13 840 | 1 443               | duży   |
| DLG7  | 574    | 43                  | mały   |
| DLG8  | 546    | 34                  | mały   |
| DLG9  | 756    | 62                  | mały   |
| DLG10 | 5 378  | 497                 | duży   |
| DLG11 | 2 024  | 121                 | średni |
| DLG12 | 5 645  | 469                 | duży   |
| DLG13 | 2 157  | 229                 | średni |
| DLG14 | 453    | 24                  | mały   |
| DLG15 | 558    | 41                  | mały   |
| DLG16 | 5 574  | 745                 | duży   |
| DLG17 | 651    | 38                  | mały   |
| DLG18 | 17 085 | 1 694               | duży   |

Skala: mały < 1 000 węzłów; średni 1 000–5 000; duży > 5 000.

## 3.3 Metodyka symulacji broken lineage przez usuwanie zadań

Ponieważ zbiór DLG-DG-23 zawiera w pełni zaobserwowane grafy lineage, zjawisko broken lineage symulowane jest przez kontrolowane usuwanie *zadań* (nie pojedynczych krawędzi). Podejście to wiernie odwzorowuje przyczynę broken lineage: zniknięcie tabeli tymczasowej lub ukrycie logiki przez UDF odpowiada utracie całego węzła-zadania wraz z jego krawędziami.

Kandydatami do usunięcia są *zadania łączące* — zadania mające co najmniej jedną wejściową i jedną wyjściową krawędź *DATA\_FLOW* ( $\text{in\_df} > 0 \wedge \text{out\_df} > 0$ ). Usunięcie takiego zadania zrywa lineage po obu stronach: tabele karmiące je tracą konsumenta, a tabele przez nie karmione — producenta. Z grafu  $G$  budowana jest instancja klasyfikacji węzłów:

1. usuwa się losowy podzbiór zadań łączących (udział regulowany parametrem `removal_ratio`; w eksperymentach przyjęto 10%, a wpływ tego parametru zbadano w analizie wrażliwości w sekcji 4.6),
2. zbiór zainfekowanych tabel to suma tabel incydentnych do usuniętych zadań,
3. graf obserwowany  $G_{\text{obs}}$  powstaje przez usunięcie wybranych zadań i ich krawędzi,
4. kandydatami są wszystkie tabele w  $G_{\text{obs}}$ ; etykieta 1 oznacza tabelę zainfekowaną, etykieta 0 — czystą.

Dodatkowo zaimplementowano wariant kontrolny, w którym każda instancja usuwa dokładnie *jedno* zadanie, co daje czystą interpretację przyczynową (jedno zdarzenie broken lineage = jedno brakujące zadanie).

### 3.3.1 Kontrola przecieku: wykluczenie węzłów izolowanych

Usunięcie zadania może pozbawić tabelę *wszystkich* krawędzi przepływu danych ( $\text{in\_df} = 0 \wedge \text{out\_df} = 0$ ). Ponieważ w oryginalnych grafach tabele izolowane nie występują, trywialna reguła „tabela izolowana  $\Rightarrow$  zainfekowana” pozwala rozpoznać znaczną część pozytywów bez żadnego uczenia, sztucznie zawyżając wynik. Aby zapewnić uczciwą ewaluację, wprowadzono flagę `exclude_isolated`, która usuwa z puli kandydatów tabele w pełni izolowane. Wariant ten stawia trudniejsze i bardziej realistyczne zadanie: wykryć tabele o *częściowo* zerwanym lineage, które nadal mają część połączeń. W pracy raportowane są oba warianty, a wariant uczciwy (z wykluczeniem izolowanych) traktowany jest jako główny.

Tabela 3.2 przedstawia skalę symulacji dla czterech wartości parametru `removal_ratio`. W 18 grafach zbioru DLG-DG-23 znajduje się łącznie 1453 zadania łączące (od 3 do 404 na graf), spośród których usuwana jest losowa część. Liczby podano dla wariantu *przed* wykluczeniem tabel izolowanych, ponieważ tylko wtedy można zmierzyć ich udział wśród pozytywów.

TABELA 3.2: Charakterystyka symulacji broken lineage w zależności od parametru `removal_ratio`. Wartości uśrednione po 18 grafach i trzech ziarnach losowości (42–44). Ostatnia kolumna — odsetek tabel zainfekowanych, które utraciły *wszystkie* krawędzie przepływu danych i są rozpoznawalne trywialną regułą izolacji.

| <code>removal_ratio</code> | Usuniętych<br>zadań | Tabel<br>zainfekowanych | Udział<br>pozytywów | Pozytywów<br>izolowanych |
|----------------------------|---------------------|-------------------------|---------------------|--------------------------|
| 0,05                       | 4,2                 | 15,1                    | 11,4%               | 19,7%                    |
| 0,10                       | 8,0                 | 25,1                    | 19,0%               | 22,9%                    |
| 0,20                       | 16,2                | 45,2                    | 34,1%               | 29,8%                    |
| 0,30                       | 24,3                | 62,6                    | 47,3%               | 34,5%                    |

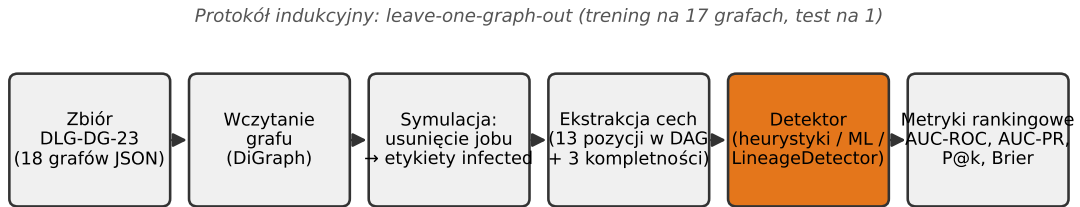
Dwie obserwacje z tabeli mają znaczenie dla konstrukcji eksperymentu. Po pierwsze, przyjęta w biegu kanonicznym wartość 0,10 daje średnio 25 tabel zainfekowanych na graf przy 19% pozytywów — zadanie pozostaje wyraźnie niezrównoważone, lecz klasa pozytywna jest dostatecznie liczna, by metryki rankingowe były wiarygodne. Po drugie, udział pozytywów trywialnych

(tabel w pełni izolowanych) rośnie z 19,7% do 34,5% wraz ze wzrostem odsetka usuwanych zadań: im silniejsze zaburzenie, tym większa część zadania daje się rozwiązać bez żadnego uczenia. Liczby te określają skalę przecieku, któremu zapobiega opisana wyżej flaga `exclude_isolated` — przy biegu kanonicznym usuwa ona z puli kandydatów blisko jedną czwartą pozytywów, i to tych najłatwiejszych.

### 3.4 Architektura rozwiązania

Potok przetwarzania składa się z czterech komponentów: modułu wczytywania grafów, modułu symulacji broken lineage (budowa instancji klasyfikacji węzłów), metod detekcji oraz modułu ewaluacji indukcyjnej.

Rysunek 3.1 przedstawia schemat potoku przetwarzania: od wczytania grafów z plików JSON, przez symulację broken lineage i ekstrakcję cech, po scoring detektora i wyznaczenie metryk rankingowych.



RYSUNEK 3.1: Schemat potoku przetwarzania. Każda instancja ewaluacji przebiega w protokole indukcyjnym leave-one-graph-out: model trenowany jest na 17 grafach, a testowany na grafie niewidzianym.

#### 3.4.1 Moduł wczytywania danych

Dane wejściowe każdego grafu przechowywane są w dwóch plikach JSON: `DLGi-node.json` (opisy węzłów z atrybutem `asset_type`) oraz `DLGi-edge.json` (krawędzie z atrybutami `relation_type` i `relation_id`). Moduł `loader.py` buduje z nich obiekt `nx.DiGraph` (NetworkX) i zwraca słownik wszystkich 18 grafów.

#### 3.4.2 Moduł symulacji (`job_removal`)

Moduł `job_removal.py` realizuje metodykę z sekcji 3.3: identyfikuje zadania łączące, usuwa ich podzbiór, wyznacza zbiór zainfekowanych tabel i zwraca instancję (graf  $G_{\text{obs}}$ , listę kandydatów, etykiety, listę usuniętych jobów). Obsługuje warianty `side` (strona incydencji liczona jako infected: producenta, konsumenta lub obie) oraz `exclude_isolated`. Funkcja `inductive_split` dzieli identyfikatory grafów na rozłączne zbiory treningowy i testowy (cross-graph).

#### 3.4.3 Moduł ewaluacji (`node_metrics`)

Moduł `node_metrics.py` wyznacza metryki rankingowe: AUC-ROC, AUC-PR, Precision@k, Recall@k, Hits@k oraz Brier score. Wyniki agregowane są po grafach testowych w protokole induk-

cyjnym; stosuje się powtórzenia z różnymi ziarnami i trzymowanie wartości skrajnych (min/max) dla stabilności oszacowań.

Ewaluację prowadzi się w protokole *indukcyjnym* cross-graph: model trenuje się na podzbiorze grafów, a testuje na grafach niewidzianych (leave-one-graph-out lub podział zbioru 18 grafów). Zbiory treningowy i testowy są rozłączne, co sprawdza zdolność metody do generalizacji na nowe grafy, a nie jedynie do zapamiętania jednego. Konsekwencją tego protokołu jest wykluczenie metody Node2Vec: uczy ona zanurzeń węzłów transduktywnie, dla konkretnego grafu, przez co embeddingi są geometrycznie nieporównywalne między grafami. Nie jest to luka, lecz *wynik*: motywuje zastosowanie indukcyjnych grafowych sieci neuronowych (GraphSAGE, R-GCN, HGT), wskazanych w rozdziale 5 jako kierunek dalszych badań.

### 3.5 Zaimplementowane metody detekcji

Zaimplementowano trzy grupy metod detekcji zainfekowanych tabel: heurystyki strukturalne, klasyczne klasyfikatory ML oraz autorski detektor LineageDetector.

#### 3.5.1 Heurystyki strukturalne

Heurystyki strukturalne przypisują tabeli wynik podejrzliwości wyłącznie na podstawie jej pozycji w grafie, bez etapu uczenia. Są naturalnym punktem odniesienia (*baseline*) dla metod uczonych. Miary oparte na wspólnych sąsiadach (Common Neighbors, Jaccard, Adamic-Adar) są w grafie dwudzielnym nieskuteczne, ponieważ węzły tego samego typu nigdy nie mają wspólnych sąsiadów bezpośrednich; wykluczono je z implementacji.

W pracy zaimplementowano cztery heurystyki bazowe:

- **Anomalia stopnia** (`degree_anomaly`) — wynik  $1/(1 + \deg(t))$ , gdzie  $\deg(t)$  jest stopniem całkowitym tabeli. Intuicja: tabela, która utraciła zadanie, ma mniej połączeń niż przed zerwaniem lineage.
- **Brzeg grafu** (`boundary`) — suma wskaźników  $1[\text{in\_df} = 0] + 1[\text{out\_df} = 0]$ . Tabele na brzegu przepływu (korzenie i liście) są podejrzane, ponieważ zerwanie lineage często wypycha tabelę na brzeg grafu.
- **Niska łączność z zadaniami** (`low_job_connectivity`) — wynik  $1/(1 + n_J(t))$ , gdzie  $n_J(t)$  to liczba unikatowych zadań sąsiadujących z tabelą przez krawędzie *DATA\_FLOW*.
- **Reguła korzenia** (`rule_root`) — binarny wskaźnik  $1[\text{in\_df} = 0]$ : każda tabela bez producenta jest uznawana za podejrzaną. To celowo trywialny punkt odniesienia, obrazujący siłę najprostszego możliwego symptomu.

Heurystyki te mierzą wyłącznie *lokalny* symptom (stopień, rola); ich ograniczenie — w szczególności nieodróżnianie osieroconej tabeli od prawdziwego źródła — motywuje metody uczone oraz autorski detektor LineageDetector (sekcja 3.5.3), który wzbogaca opis węzła o relacyjny *score kompletności* (peer-consistency).

#### 3.5.2 Klasyczne metody uczenia maszynowego

Klasyczne metody ML traktują detekcję jako binarną klasyfikację węzła: wejściem jest wektor cech opisujących pozycję tabeli w grafie DAG, a wyjściem prawdopodobieństwo zainfekowania. Ze względu na anonimizację (UUID) jako cechy przyjęto wyłącznie atrybuty topologiczne — metryki oparte na nazwach są nieaplikowalne.

## Wektor cech węzła

Tabela 3.3 przedstawia pełny wektor 13 cech topologicznych wyznaczanych dla każdej tabeli-kandydata. Cechy opisują pozycję węzła w grafie przepływu danych — to ona, a nie sam stopień, pozwala odróżnić prawdziwą tabelę źródłową od tabeli osieroconej po usunięciu zadania.

TABELA 3.3: Wektor cech topologicznych węzła (moduł `node_features`).

| Nr | Cecha                         | Opis                                                             | Rola w detekcji                                          |
|----|-------------------------------|------------------------------------------------------------------|----------------------------------------------------------|
| 1  | <code>in_df</code>            | liczba wejściowych krawędzi <i>DATA_FLOW</i> (od zadań)          | brak producenta (= 0) to główny symptom zerwania         |
| 2  | <code>out_df</code>           | liczba wyjściowych krawędzi <i>DATA_FLOW</i> (do zadań)          | symetryczny symptom: utrata konsumenta                   |
| 3  | <code>degree</code>           | stopień całkowity węzła (wszystkie typy krawędzi)                | ogólna łączność; spada po usunięciu zadania              |
| 4  | <code>role</code>             | rola w przepływie: źródło / pośrednia / ujście / poza przepływem | pozycja funkcjonalna tabeli w DAG                        |
| 5  | <code>flow_depth</code>       | głębokość w DAG (odległość od źródła)                            | osierocona tabela ma sztucznie małą głębokość            |
| 6  | <code>flow_reach</code>       | zasięg w dół DAG (odległość do ujść)                             | symetrycznie: sztucznie mały zasięg                      |
| 7  | <code>depth_plus_reach</code> | suma głębokości i zasięgu                                        | długość ścieżki przez węzeł; wykrywa „skrótowe” łańcuchy |
| 8  | <code>is_root</code>          | wskaźnik <code>1[in_df = 0]</code>                               | binarna wersja symptomu utraty producenta                |
| 9  | <code>is_leaf</code>          | wskaźnik <code>1[out_df = 0]</code>                              | binarna wersja symptomu utraty konsumenta                |
| 10 | <code>n_field_children</code> | liczba pól tabeli (krawędzie <i>PARENT_CHILD</i> )               | rozmiar tabeli; sygnał niezależny od przepływu           |
| 11 | <code>n_job_neighbors</code>  | liczba unikatowych zadań sąsiadujących                           | łączność z warstwą zadań                                 |
| 12 | <code>avg_job_in_df</code>    | średni stopień wejściowy sąsiednich zadań                        | charakterystyka otoczenia węzła                          |
| 13 | <code>avg_job_out_df</code>   | średni stopień wyjściowy sąsiednich zadań                        | j.w. — od strony wyjść                                   |

Detektor LineageDetector (sekcja 3.5.3) rozszerza ten wektor o trzy cechy kompletności (`peer_producer_score`, `peer_consumer_score`, `completeness_anomaly`), opisane w sekcji 3.5.

## Random Forest

Random Forest (RF) to metoda uczenia zespołowego oparta na uśrednianiu predykcji drzew decyzyjnych trenowanych na losowych podpróbkach danych i cech (*bagging*). Dla obsługi niezrównoważonych klas zastosowano `class_weight="balanced"`.

## RUSBoost

RUSBoost (*Random Under-Sampling Boosting*) łączy boosting AdaBoost z losowym podpróbkowaniem klasy większościowej przed każdą iteracją. Zaimplementowano go dla bezpośredniego nawiązania do wyników Boińskiego et al. [5], przy czym w niniejszej pracy klasyfikuje on węzły, a nie pary obiektów.

## LightGBM

LightGBM to implementacja gradientowego boostingu drzew decyzyjnych, budująca kolejne drzewa sekwencyjnie, każde minimalizując błąd reszkowy poprzednika. Parametr `scale_pos_weight` (lub `class_weight`) kompensuje niebalansowanie klas.



### 3.5.3 Autorski detektor — LineageDetector

Autorski detektor `LineageDetector` (klasa `LineageHealthDetector`) łączy trzy elementy w jeden skalibrowany model:

1. **cechy pozycji w DAG** (z modułu `node_features`) — lokalny opis węzła,
2. **score kompletności peer-consistency** (opisany poniżej) — sygnał relacyjny stanowiący sedno wkładu własnego pracy,
3. **kalibrację prawdopodobieństw** metodą izotoniczną (`CalibratedClassifierCV`) — wyjście jest rzetelnym  $p$ (zainfekowana), a nie dowolnym wynikiem.

**Score kompletności — peer-consistency.** Zaproponowany w pracy score kompletności opiera się na założeniu, że tabela o zerwanym lineage jest *strukturalnie niespójna* ze swoimi sąsiadami (peerami) w grafie DAG. Dla tabeli-korzenia ( $\text{in\_df} = 0$ ) rozważa się zbiór peerów-wejść — inne tabele wpadające do tych samych zadań downstream. Jeśli peerzy *mają* producenta, a rozpatrywana tabela nie, to jej brak producenta jest podejrzany: prawdziwe źródło danych sąsiaduje zwykle z innymi źródłami. Symetrycznie dla tabeli-liścia ( $\text{out\_df} = 0$ ) i jej peerów po stronie konsumpcji. Formalnie dla tabeli  $t$  definiuje się:

$$\text{prod}(t) = \frac{1}{|P(t)|} \sum_{p \in P(t)} \mathbf{1}[\text{in\_df}(p) > 0], \quad \text{cons}(t) = \frac{1}{|C(t)|} \sum_{p \in C(t)} \mathbf{1}[\text{out\_df}(p) > 0],$$

gdzie  $P(t)$  i  $C(t)$  oznaczają zbiory peerów odpowiednio po stronie producenta i konsumenta, a  $\mathbf{1}[\cdot]$  — funkcję wskaźnikową. Bramkowany sygnał anomalii aktywuje się wyłącznie na brzegu grafu:

$$\text{anomaly}(t) = \max(\mathbf{1}[\text{in\_df}(t) = 0] \cdot \text{prod}(t), \mathbf{1}[\text{out\_df}(t) = 0] \cdot \text{cons}(t)).$$

Sygnał ten ma charakter relacyjny — jest jakościowo różny od lokalnego stopnia węzła i dostarcza informacji niedostępnej klasyfikatorowi korzystającemu wyłącznie z cech lokalnych. Definicja jest symetryczna względem obu stron przepływu, jednak na zbiorze DLG-DG-23 aktywna okazuje się wyłącznie strona producenta: każde zadanie w tym zbiorze zapisuje dokładnie jedną tabelę, przez co zbiór  $C(t)$  jest zawsze pusty. Konsekwencje tej własności omówiono w sekcji 4.7. Trzy wynikające z niego cechy (`peer_producer_score`, `peer_consumer_score`, `completeness_anomaly`) rozszerzają wektor cech z tabeli 3.3 i są dostępne wyłącznie detektorowi `LineageDetector`.

Bazowym estymatorem jest Random Forest (domyślnie; wariant alternatywny — `LightGBM`). Interfejs `fit/score` jest zgodny z pozostałymi klasyfikatorami, co umożliwia bezpośrednie porównanie w tym samym potoku. Konstrukcja detektora umożliwia badania ablacyjne: wyłączenie kalibracji (`calibrate=False`) izoluje jej wpływ na Brier score, a wyłączenie cech kompletności (`use_completeness=False`) — wpływ sygnału relacyjnego na AUC.

## 3.6 Implementacja

Sekcja opisuje realizację techniczną potoku z sekcji 3.4: zastosowane narzędzia, organizację kodu oraz te fragmenty implementacji, które są istotne dla zrozumienia metody, a nie wynikają wprost z jej opisu formalnego.

### 3.6.1 Stos technologiczny

Całość zaimplementowano w języku Python. Tabela 3.4 zestawia wykorzystane biblioteki wraz z wersjami, w których wykonano bieg kanoniczny. Podanie wersji jest istotne dla odtwarzalności: implementacje algorytmów uczenia maszynowego bywają modyfikowane między wydaniem, co może wpływać na wyniki liczbowe.

TABELA 3.4: Stos technologiczny i wersje bibliotek użyte w biegu kanonicznym.

| Składnik         | Wersja | Rola w pracy                                   |
|------------------|--------|------------------------------------------------|
| Python           | 3.10.2 | środowisko uruchomieniowe                      |
| NetworkX         | 3.4.2  | reprezentacja i przetwarzanie grafów           |
| NumPy            | 1.26.4 | operacje macierzowe, wektory cech              |
| pandas           | 1.5.1  | agregacja wyników, eksport CSV                 |
| scikit-learn     | 1.6.1  | Random Forest, kalibracja izotoniczna, metryki |
| LightGBM         | 4.6.0  | gradientowy boosting drzew                     |
| imbalanced-learn | 0.14.1 | RUSBoost                                       |
| SciPy            | 1.15.3 | test kolejności par Wilcoxona                  |
| Matplotlib       | 3.8.0  | rysunki                                        |
| pytest           | 9.0.2  | testy jednostkowe                              |

### 3.6.2 Organizacja repozytorium

Kod podzielono na pakiety odpowiadające etapom potoku; skrypty uruchomieniowe oddzielono od modułów bibliotecznych, aby każdy eksperyment z rozdziału 4 dawał się odtworzyć jednym poleceniem. Listing 3.1 przedstawia strukturę katalogów.

```
src/
  data/
    loader.py           # wczytanie 18 grafow DLG-DG-23 z JSON
    dutkiewicz_adapter.py # adapter benchmarku zewnetrznego (graf DUT)
  detection/
    job_removal.py       # symulacja broken lineage, etykiety, podzial
    node_features.py     # 13 cech topologicznych wezla
    completeness.py      # score peer-consistency (wkład własny)
    lineage_detector.py   # LineageHealthDetector
    node_classifier.py    # heurystyki + klasyczne klasyfikatory
    node_metrics.py       # metryki rankingowe
    run_detection_experiments.py # bieg kanoniczny (rozdz. 4.2)
    run_ablation.py       # badanie ablacyjne (rozdz. 4.5)
    run_cross_dataset.py  # test cross-dataset (rozdz. 4.7)
    run_removal_sensitivity.py # analiza wzrazliwosci (rozdz. 4.6)
  scripts/
    test_istotnosci.py    # testy Wilcoxona (rozdz. 4.4)
    tabela_symulacji.py   # tabela charakterystyki symulacji
  tests/                  # 83 testy jednostkowe (pytest)
  results/                # wyniki biegow w formacie CSV
```

LISTING 3.1: Struktura repozytorium (fragmenty istotne dla detekcji węzłów).

### 3.6.3 Format danych i wczytywanie

Każdy graf zbioru DLG-DG-23 opisany jest parą plików JSON: lista węzłów z atrybutem `asset_type` oraz lista krawędzi z atrybutami `relation_id` i `relation_type` (listing 3.2). Moduł `loader.py` przekształca je w obiekt `nx.DiGraph`, przenosząc typ węzła i typ relacji do atrybutów grafu — dzięki temu pozostałe moduły filtrują podgraf przepływu danych bez ponownego sięgania do plików.

```
// DLG1-node.json
{"nodes": [
  {"asset_id": "3f2a...c81", "asset_type": "Data Table"},
  {"asset_id": "9b07...4de", "asset_type": "Data Job"},
  {"asset_id": "a154...77f", "asset_type": "Data Field"}
]}

// DLG1-edge.json
{"edges": [
  {"relation_id": "r-0012", "relation_type": "DATA_FLOW",
   "source": "3f2a...c81", "target": "9b07...4de"},
  {"relation_id": "r-0013", "relation_type": "PARENT_CHILD",
   "source": "3f2a...c81", "target": "a154...77f"}
]}
```

LISTING 3.2: Fragment plików wejściowych DLGi-node.json i DLGi-edge.json (identyfikatory skrócone).

### 3.6.4 Wyznaczanie score'u peer-consistency

Najistotniejszym fragmentem implementacji jest wyznaczanie relacyjnego score'u kompletności, zdefiniowanego wzorami w sekcji 3.5.3. Listing 3.3 pokazuje jego rdzeń. Warto zwrócić uwagę na dwa szczegóły, których sam wzór nie oddaje. Po pierwsze, zbiory peerów budowane są *przez zadania*: peerami tabeli po stronie producenta są inne tabele wpadające do tych samych zadań downstream — w grafie dwudzielnym nie istnieje bowiem bezpośrednia krawędź tabela–tabela. Po drugie, sygnał anomalii jest *bramkowany*: aktywuje się wyłącznie wtedy, gdy rozpatrywana tabela sama znajduje się na brzegu przepływu (`in_df = 0` lub `out_df = 0`). Bez tego bramkowania score mierzyłby jedynie własności otoczenia, nie zaś niespójność tabeli z jej otoczeniem.

```
1 # Strona producenta: peerzy = inne wejścia do downstream-jobow tej tabeli.
2 downstream_jobs = [v for _, v, d in G.out_edges(t, data=True)
3                     if d.get("relation_type") == "DATA_FLOW"
4                     and G.nodes[v].get("asset_type") == "DataJob"]
5 prod_peers = set()
6 for j in downstream_jobs:
7     prod_peers.update(job_inputs.get(j, ()))
8 prod_peers.discard(t)
9 prod_score = (np.mean([1.0 if in_df[p] > 0 else 0.0 for p in prod_peers])
10              if prod_peers else 0.0)
11
12 # Bramkowana anomalia: liczy się tylko na brzegu (korzen/licz).
13 anomaly = 0.0
14 if in_df[t] == 0:
15     anomaly = max(anomaly, prod_score)
```

```

16 if out_df[t] == 0:
17     anomaly = max(anomaly, cons_score)

```

LISTING 3.3: Rdzeń wyznaczania score'u peer-consistency (moduł `completeness.py`).

### 3.6.5 Protokół indukcyjny w kodzie

Pętla ewaluacji realizuje schemat *leave-one-graph-out*: dla każdego grafu testowego model uczy się na wszystkich pozostałych. Listing 3.4 przedstawia jej szkielec. Kluczowa dla poprawności protokołu jest kolejność operacji: instancje wszystkich grafów budowane są *przed* pętlą, przy wspólnym ziarnie, a pula treningowa powstaje przez wykluczenie grafu testowego z gotowego słownika — model nie ma więc dostępu do żadnej informacji z grafu testowego, także pośrednio przez dobór usuwanych zadań.

```

1 datasets = build_datasets(graphs, removal_ratio, seed, side,
2                           exclude_isolated=exclude_isolated)
3 for test_g in datasets:
4     ds_test = datasets[test_g]
5     train_pool = [datasets[g] for g in datasets if g != test_g]
6
7     for name in FIT_MODELS:
8         clf = _make_model(name, seed).fit(train_pool)
9         m = node_detection_metrics(ds_test["labels"], clf.score(ds_test))
10        rows.append({"graph": test_g, "algorithm": name, **m})

```

LISTING 3.4: Szkielec pętli *leave-one-graph-out* (moduł `run_detection_experiments.py`).

Wyniki z kolejnych ziaren agregowane są średnią z trymowaniem wartości skrajnych (odrzućenie najmniejszej i największej), co ogranicza wpływ pojedynczego nietypowego losowania na oszacowanie.

### 3.6.6 Złożoność obliczeniowa

Koszt przetwarzania jednego grafu jest liniowy względem liczby krawędzi. Wyznaczenie stopni *DATA\_FLOW*, ekstrakcja cech oraz budowa zbiorów peerów wymagają stałej liczby przejść po krawędziach, a metadane przepływu (głębokość i zasięg w DAG) wyznaczane są przeszukiwaniem grafu, również w czasie  $O(|V|+|E|)$ . Dominującym składnikiem kosztu nie jest zatem przetwarzanie grafu, lecz trenowanie klasyfikatorów: w protokole *leave-one-graph-out* model uczony jest od nowa dla każdego grafu testowego, co przy 18 grafach, trzech ziarnach i czterech wariantach detektora daje 216 pełnych treningów. Bieg ablacyjny z sekcji 4.5 zajął z tego powodu 412 sekund, podczas gdy sama ekstrakcja cech dla wszystkich 18 grafów trwa poniżej sekundy. Skala zbioru DLG-DG-23 (do 17 085 węzłów na graf) nie stanowi więc ograniczenia wydajnościowego.

## 3.7 Testy jednostkowe i odtwarzalność

### 3.7.1 Zakres testów

Poprawność implementacji zweryfikowano zestawem 83 testów jednostkowych uruchamianych narzędziem `pytest`; wszystkie przechodzą. Tabela 3.5 przedstawia ich rozkład na moduły.

TABELA 3.5: Rozkład testów jednostkowych na moduły.

| Plik testowy                            | Liczba    | Weryfikowane własności                                                                                                                         |
|-----------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_loader.py</code>             | 29        | poprawność typów węzłów i krawędzi, zgodność liczby obiektów z plikami źródłowymi, wczytanie wszystkich 18 grafów                              |
| <code>test_splitter.py</code>           | 17        | rozłączność podziałów, zgodność liczności podzbiorów                                                                                           |
| <code>test_job_removal.py</code>        | 12        | identyfikacja zadań łączących, poprawność etykiet, rozłączność zbiorów treningowego i testowego, działanie flagi <code>exclude_isolated</code> |
| <code>test_scenario_splitter.py</code>  | 9         | scenariusze usuwania krawędzi (badanie wstępne)                                                                                                |
| <code>test_dutkiewicz_adapter.py</code> | 6         | mapowanie benchmarku zewnętrznego na model tabela-zadanie- <code>DATA_FLOW</code>                                                              |
| <code>test_node_features.py</code>      | 4         | wymiar wektora cech, zgodność z <code>feature_names()</code> , wartości brzegowe dla węzłów izolowanych                                        |
| <code>test_completeness.py</code>       | 3         | poprawność score’u peer-consistency, bramkowanie sygnału anomalii                                                                              |
| <code>test_lineage_detector.py</code>   | 3         | interfejs <code>fit/score</code> , zakres zwracanych prawdopodobieństw, warianty ablacyjne                                                     |
| <b>Razem</b>                            | <b>83</b> |                                                                                                                                                |

Testy sprawdzają nie tylko poprawność techniczną, lecz także dwie własności istotne metodycznie. Pierwsza to *brak przecieku* między zbiorem treningowym a testowym: test weryfikuje, że identyfikatory grafów w obu zbiorach są rozłączne. Druga to *determinizm*: przy ustalonym ziarnie losowości powtórne wykonanie symulacji daje identyczny zbiór usuniętych zadań i identyczne etykiety — bez tego liczby raportowane w rozdziale 4 nie byłyby odtwarzalne.

### 3.7.2 Odtwarzalność wyników

Wszystkie liczby przedstawione w rozdziale 4 pochodzą z jednego *biegu kanonicznego* o ustalonej konfiguracji: 18 grafów, `removal_ratio = 0,10`, `side = both`, trzy powtórzenia z ziarnami 42–44, protokół indukcyjny `leave-one-graph-out`, próg wiarygodności `MIN_POS = 5`, wariant ucziwy (`exclude_isolated`). Wyniki pośrednie zapisywane są w postaci plików CSV w katalogu `results/`, osobno dla wariantu ucziwego i napompowanego, wraz z opisem konfiguracji. Eksperymenty poboczne (ablacja, analiza wrażliwości, test cross-dataset) mają własne skrypty i własne pliki wynikowe, wymienione w listingu 3.1, co pozwala odtworzyć każdą tabelę rozdziału 4 niezależnie od pozostałych.

## 3.8 Dostępność implementacji i pochodzenie algorytmów

Całość implementacji opisanej w niniejszej pracy — moduły wczytywania danych, symulacji broken lineage, ekstrakcji cech, metod detekcji oraz ewaluacji, wraz z testami jednostkowymi i skryptami odtwarzającymi eksperymenty z rozdziału 4 — udostępniono w publicznym repozytorium Git pod adresem <https://github.com/Catcuss/magisterka-data-lineage><sup>1</sup>. Repozytorium zawiera instrukcję uruchomienia (dodatek A) oraz pliki konfiguracyjne biegu kanonicznego, co zapewnia odtwarzalność raportowanych wyników.

Metody z grupy heurystyk oraz klasyczne klasyfikatory nie są oryginalnym wkładem pracy — stanowią *punkty odniesienia* (baseline) zaczerpnięte z literatury i z publicznych bibliotek, a wkładem własnym jest ich *adaptacja* do zadania detekcji zainfekowanych węzłów w grafie lineage

<sup>1</sup>Dostęp zweryfikowano 19 sierpnia 2026 r.

oraz autorski detektor LineageDetector. Pochodzenie i zakres adaptacji poszczególnych składników zestawiono poniżej:

- **Heurystyki strukturalne** — inspirowane rodziną miar predykcji krawędzi w grafach (Common Neighbors, Jaccard, Adamic–Adar, Preferential Attachment) [25]. Ponieważ graf przepływu danych jest dwudzielny, miary oparte na wspólnych sąsiadach są w nim nieskuteczne; zaadaptowano je do postaci *scoringu pojedynczego węzła* opartego na stopniu i roli w DAG.
- **Random Forest** — implementacja z biblioteki `scikit-learn` [34]. Adaptacja: wejściem jest autorski wektor cech topologicznych węzła (tab. 3.3) zamiast cech pary obiektów; niezbalansowanie klas obsłużono przez `class_weight`.
- **RUSBoost** — implementacja z biblioteki `imbalanced-learn` [28], wybrana dla bezpośredniego nawiązania do pracy Boińskiego et al. [5]. Adaptacja: klasyfikacja *węzła* zamiast pary, na cechach topologicznych.
- **LightGBM** — biblioteka gradientowego boostingu firmy Microsoft [23]. Adaptacja: strojenie `scale_pos_weight` pod rozkład klas w zadaniu detekcji węzłów.
- **LineageDetector** — *wkład własny*: łączy powyższe cechy z autorskim relacyjnym score kompletności peer-consistency (sekcja 3.5.3) i kalibracją izotoniczną; nie ma bezpośredniego odpowiednika w literaturze.

Takie rozgraniczenie — komponenty zapożyczone z jawnie wskazanym źródłem kontra elementy autorskie — pozwala jednoznacznie ocenić rzeczywisty wkład pracy i odróżnić go od narzędzi ogólnego przeznaczenia.

## Rozdział 4

# Ocena rozwiązania

W rozdziale przedstawiono wyniki eksperymentów przeprowadzonych zgodnie z metodyką opisaną w rozdziale 3. Dla każdej z metod detekcji podano wartości metryk rankingowych uśrednione po grafach zbioru DLG-DG-23 w protokole indukcyjnym.

### 4.1 Badanie wstępne: predykcja krawędzi

Punktem wyjścia pracy było ujęcie broken lineage jako *predykcji brakujących krawędzi* — zgodne z dotychczasową literaturą. W badaniu wstępnym każdy z 18 grafów poddano trzem scenariuszom usuwania krawędzi (oznaczonym A, B, C, różniącym się sposobem doboru krawędzi usuwanych i próbkowaniem negatywów), a następnie oceniono zdolność metod do odtworzenia usuniętych powiązań. Testowano heurystyki predykcji krawędzi wywodzące się z analizy sieci złożonych [29, 31] — Preferential Attachment [4], L3 [26], Katz oraz Personalized PageRank — klasyczne klasyfikatory (Random Forest, RUSBoost, LightGBM) oraz zanurzenia sieciowe (Node2Vec [17] + MLP), w tym samym protokole indukcyjnym cross-graph.

Wyniki okazały się niesatysfakcjonujące i niestabilne. Na grafach wiarygodnych (dostatecznie liczna klasa pozytywna w zbiorze testowym) średnie AUC-ROC utrzymywało się w pobliżu poziomu losowego (0,5), przy bardzo wysokiej wariancji między grafami i scenariuszami. Jedynie pojedyncze pary (graf, scenariusz) osiągały AUC-ROC rzędu 0,70, lecz bez powtarzalności — żadna metoda nie generalizowała się w sposób spójny na grafy niewidziane. Obserwacja ta jest zbieżna z wynikami Dutkiewicza et al. [15], u których scenariusz tabel tymczasowych — najczęstsza praktyczna przyczyna broken lineage — osiągał AUROC 0,49, czyli poziom losowy, nawet dla zaawansowanych metod grafowo-semantycznych.

Przyczyną jest *rzadkość* grafów DLG-DG-23: liczba krawędzi przepływu danych jest o rząd wielkości mniejsza od liczby węzłów (tab. 3.1), przez co sygnał o konkretnej brakującej parze obiektów jest zbyt słaby, aby umożliwić wiarygodną predykcję krawędzi bez dostępu do semantyki nazw. Wynik ten stanowi bezpośrednią motywację *zmiany ujęcia problemu* na poziom węzłów (sekcja 2.5): zamiast pytać, *która krawędź* została utracona, pyta się, *które tabele* są dotknięte brakiem — zadanie słabiej uzależnione od pojedynczej krawędzi, a przez to wykonalne mimo rzadkości grafu.

### 4.2 Wyniki detekcji węzłów o zerwanym lineage

Tabela 4.1 przedstawia średnie metryki rankingowe wszystkich metod detekcji w protokole indukcyjnym, w wariancie uczciwym (z wykluczeniem węzłów izolowanych, sekcja 3.3.1), uśred-

nione po ośmiu grafach wiarygodnych. Wariant ten przyjęto jako główny, ponieważ nie korzysta z trywialnego przecieku „tabela izolowana  $\Rightarrow$  zainfekowana” (por. sekcja 4.6).

TABELA 4.1: Średnie metryki detekcji zainfekowanych tabel per metoda (protokół indukcyjny, wariant uczciwy), wraz z odchyleniem standardowym po ośmiu grafach wiarygodnych; gwiazdka — wkład własny, najlepsza wartość w kolumnie pogrubiona.

| Metoda                                      | AUC-ROC                  | AUC-PR                   | P@k                      | Brier                    |
|---------------------------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| <i>Heurystyki strukturalne</i>              |                          |                          |                          |                          |
| Anomalia stopnia                            | 0,427 $\pm$ 0,045        | 0,165 $\pm$ 0,052        | 0,142 $\pm$ 0,070        | 0,134 $\pm$ 0,026        |
| Brzeg (root/leaf)                           | 0,455 $\pm$ 0,046        | 0,137 $\pm$ 0,028        | 0,146 $\pm$ 0,038        | 0,745 $\pm$ 0,088        |
| Niska łączność zadań                        | 0,324 $\pm$ 0,069        | 0,112 $\pm$ 0,026        | 0,094 $\pm$ 0,055        | 0,226 $\pm$ 0,016        |
| Reguła korzenia                             | 0,574 $\pm$ 0,062        | 0,168 $\pm$ 0,025        | 0,180 $\pm$ 0,120        | 0,469 $\pm$ 0,071        |
| <i>Klasyczne metody uczenia maszynowego</i> |                          |                          |                          |                          |
| Random Forest                               | 0,749 $\pm$ 0,064        | 0,425 $\pm$ 0,085        | 0,380 $\pm$ 0,131        | 0,117 $\pm$ 0,024        |
| RUSBoost                                    | 0,631 $\pm$ 0,040        | 0,249 $\pm$ 0,036        | 0,340 $\pm$ 0,061        | 0,152 $\pm$ 0,048        |
| LightGBM                                    | 0,720 $\pm$ 0,077        | 0,381 $\pm$ 0,065        | 0,398 $\pm$ 0,059        | 0,147 $\pm$ 0,022        |
| <i>Autorski detektor</i>                    |                          |                          |                          |                          |
| LineageDetector*                            | <b>0,771</b> $\pm$ 0,056 | <b>0,435</b> $\pm$ 0,049 | <b>0,443</b> $\pm$ 0,068 | <b>0,106</b> $\pm$ 0,020 |

W przyjętym biegu  $k$  równa się liczbie tabel faktycznie zainfekowanych w grafie testowym, przez co Precision@k, Recall@k i Hits@k przyjmują tę samą wartość; w tabeli podano zatem jedną kolumnę (P@k), a relację tę opisano słownie, aby nie sugerować trzech niezależnych metryk.

LineageDetector uzyskał najwyższą wartość średnią na wszystkich czterech metrykach jednocześnie. Skala tej przewagi jest jednak bardzo różna w zależności od punktu odniesienia: nad LightGBM wynosi +0,051 AUC-ROC i +0,054 AUC-PR, lecz nad Random Forest już tylko +0,022 i +0,010 — czyli mniej niż połowę odchylenia standardowego obu metod. Sama różnica średnich nie wystarcza zatem do orzeczenia o przewadze; jej istotność statystyczną zbadano w sekcji 4.4, a udział poszczególnych składników detektora w tym wyniku — w badaniu ablacyjnym w sekcji 4.5.

Heurystyki strukturalne oparte na lokalnym symptomie wypadają słabo: żadna nie przekracza AUC-ROC 0,58, a trzy z czterech plasują się poniżej poziomu losowego. Wynika to z natury zadania w wariancie uczciwym — po wykluczeniu tabel w pełni izolowanych symptom lokalny (niski stopień, pozycja na brzegu) przestaje wystarczać, ponieważ osierocona tabela-następnik jest strukturalnie nieodróżnialna od prawdziwej tabeli źródłowej (sekcja 3.1). Rozróżnienie tych przypadków wymaga sygnału relacyjnego oraz uczenia indukcyjnego — obecnych dopiero w klasycznych klasyfikatorach i w LineageDetectorze.

Relacyjny score kompletności — sedno wkładu LineageDetektora — użyty samodzielnie jako heurystyka osiąga AUC-ROC 0,559, AUC-PR 0,206 i P@k 0,240, a więc wyraźnie powyżej pozostałych heurystyk lokalnych po stronie AUC-PR i P@k. Potwierdza to, że informacja relacyjna (spójność tabeli z jej peerami) jest jakościowo bogatsza od samego stopnia węzła; jej pełną wartość realizuje jednak dopiero LineageDetector, łącząc ją z cechami pozycji w DAG i kalibracją.

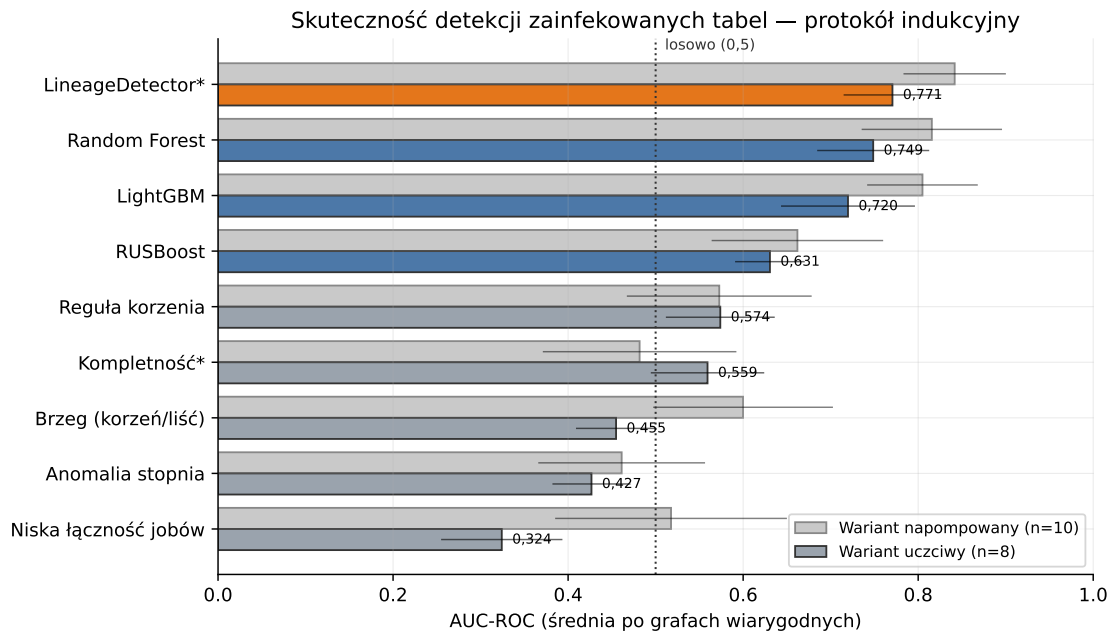
### 4.3 Wyniki w rozbiciu na poszczególne grafy

Średnie z tabeli 4.1 ukrywają rozrzut między grafami, który jest w tym zadaniu istotny — a zarazem stanowi podstawę sparowanego testu z sekcji 4.4. Tabela 4.2 przedstawia AUC-ROC osobno dla każdego z ośmiu grafów wiarygodnych, wraz z liczbą kandydatów i liczbą tabel zainfekowanych.

Rozbicie ujawnia trzy rzeczy niewidoczne w średnich.

Po pierwsze, **rozrzut jest duży w stosunku do różnic między metodami**. Wynik LineageDetektora waha się od 0,692 (DLG13) do 0,869 (DLG12) — rozpiętość 0,18 jest ośmiokrotnie





\* wkład własny. Warianty uśrednione po różnych zbiorach grafów wiarygodnych (uczciwy 8, napompowany 10) — nie odejmować 1:1.

RYSUNEK 4.1: Porównanie AUC-ROC metod detekcji w wariancie uczciwym (z wykluczeniem węzłów izolowanych) i napompowanym. Metody uczone zachowują przewagę w obu wariantach; heurystyki lokalne tracą najwięcej po usunięciu przecieku izolacji.

TABELA 4.2: AUC-ROC w rozbiściu na grafy testowe (protokół indukcyjny, wariant uczciwy). Kand. — liczba tabel-kandydatów, Zainf. — liczba tabel zainfekowanych; najlepsza wartość w wierszu pogrubiona.

| Graf           | Kand. | Zainf. | Reguła korzenia | Score kompl. | Random Forest | RUS-Boost | Light-GBM | Lineage-Detector* |
|----------------|-------|--------|-----------------|--------------|---------------|-----------|-----------|-------------------|
| DLG5           | 66    | 9      | 0,590           | 0,567        | 0,662         | 0,703     | 0,633     | <b>0,745</b>      |
| DLG6           | 568   | 82     | 0,567           | 0,627        | 0,745         | 0,619     | 0,707     | <b>0,790</b>      |
| DLG10          | 213   | 36     | 0,544           | 0,428        | 0,681         | 0,586     | 0,588     | <b>0,714</b>      |
| DLG11          | 65    | 6      | 0,642           | 0,492        | 0,732         | 0,658     | 0,776     | <b>0,804</b>      |
| DLG12          | 197   | 26     | 0,457           | 0,585        | 0,855         | 0,604     | 0,837     | <b>0,869</b>      |
| DLG13          | 100   | 13     | 0,678           | 0,602        | <b>0,723</b>  | 0,674     | 0,697     | 0,692             |
| DLG16          | 246   | 44     | 0,565           | 0,626        | <b>0,841</b>  | 0,599     | 0,783     | 0,818             |
| DLG18          | 625   | 110    | 0,549           | 0,547        | <b>0,750</b>  | 0,601     | 0,737     | 0,733             |
| <i>średnia</i> |       |        | 0,574           | 0,559        | 0,749         | 0,631     | 0,720     | <b>0,771</b>      |

większa niż jego przewaga nad Random Forest w średniej (0,022). Zmienność między grafami dominiuje więc nad zmiennością między metodami uczonymi, co bezpośrednio tłumaczy brak istotności statystycznej stwierdzony w sekcji 4.4.

Po drugie, **przewaga detektora nie jest jednorodna**. LineageDetector wypada najlepiej na pięciu z ośmiu grafów, na pozostałych trzech (DLG13, DLG16, DLG18) ustępuje Random Forest — za każdym razem niewiele, o 0,017–0,031. Nie widać przy tym systematycznej zależności od rozmiaru grafu: detektor wygrywa zarówno na najmniejszym (DLG11, 65 kandydatów), jak i na jednym z największych (DLG6, 568), a przegrywa na grafie średnim (DLG13, 100) i największym (DLG18, 625).

Po trzecie, **uporządkowanie klas metod jest natomiast stabilne**. Na każdym z ośmiu grafów obie najlepsze metody uczone (Random Forest i LineageDetector) przewyższają obie przedsta-

wione heurystyki, a różnica wynosi od 0,014 (DLG13) do 0,270 (DLG12), przy medianie około 0,13. Wniosek o przewadze uczenia indukcyjnego nad regułami lokalnymi nie zależy więc od doboru grafu testowego — w przeciwieństwie do porównań wewnątrz grupy metod uczonych.

Udział tabel zainfekowanych wśród kandydatów wynosi średnio 14,5% i jest zbliżony we wszystkich grafach (od 9% dla DLG11 do 18% dla DLG16), zadanie jest zatem porównywalnie niezrównoważone niezależnie od skali grafu.

#### 4.4 Istotność statystyczna różnic

Różnice średnich z tabeli 4.1 są w części przypadków mniejsze od odchyłeń standardowych, dlatego zbadano ich istotność statystyczną. Zastosowano test kolejności par Wilcozona [40] (*signed-rank*) w wariancie dokładnym i dwustronnym, sparowany po *grafach* testowych — zgodnie z zaleceniem Demšara [13] dla porównywania klasyfikatorów na wielu zbiorach danych. Jednostką obserwacji jest graf wiarygodny, a nie pojedynczy węzeł: pary są zależne (ten sam graf, ten sam podział indukcyjny), co wyklucza testy zakładające niezależność obserwacji.

Poprawkę Holma [20] na wielokrotne porównania zastosowano *w obrębie rodziny hipotez*, wyróżniając dwie rodziny odpowiadające dwóm pytaniom pracy: porównanie z heurystykami (5 testów) oraz porównanie z klasycznymi klasyfikatorami (3 testy). Podział ten jest podyktowany mocą testu: przy ośmiu parach najniższe osiągalne  $p$  wynosi  $2/2^8 = 0,0078$ , więc korekta na wszystkich ośmiu porównaniach naraz podniosłaby próg do 0,0625 — przy takim planie *żadna* różnica nie mogłaby zostać uznana za istotną, niezależnie od danych. Wyniki zestawia tabela 4.3.

TABELA 4.3: Istotność statystyczna różnic między LineageDetectorem a pozostałymi metodami. Test kolejności par Wilcozona (dokładny, dwustronny), sparowany po 8 grafach wiarygodnych; poprawka Holma w obrębie rodziny hipotez i metryki.  $\Delta$  — mediana różnicy na korzyść LineageDetector.

| Porównanie z         | AUC-ROC  |        | AUC-PR   |        | P@k      |        | Brier    |        |
|----------------------|----------|--------|----------|--------|----------|--------|----------|--------|
|                      | $\Delta$ | $p$    | $\Delta$ | $p$    | $\Delta$ | $p$    | $\Delta$ | $p$    |
| <i>Heurystyki</i>    |          |        |          |        |          |        |          |        |
| Anomalia stopnia     | +0,344   | 0,039* | +0,273   | 0,039* | +0,297   | 0,039* | +0,029   | 0,039* |
| Brzeg (root/leaf)    | +0,315   | 0,039* | +0,279   | 0,039* | +0,297   | 0,039* | +0,633   | 0,039* |
| Niska łączność zadań | +0,466   | 0,039* | +0,310   | 0,039* | +0,352   | 0,039* | +0,119   | 0,039* |
| Reguła korzenia      | +0,177   | 0,039* | +0,253   | 0,039* | +0,250   | 0,039* | +0,344   | 0,039* |
| Score kompletności   | +0,189   | 0,039* | +0,223   | 0,039* | +0,193   | 0,039* | +0,040   | 0,039* |
| <i>Metody uczone</i> |          |        |          |        |          |        |          |        |
| Random Forest        | +0,023   | 0,250  | +0,014   | 0,461  | +0,064   | 0,188  | +0,012   | 0,109  |
| RUSBoost             | +0,139   | 0,023* | +0,172   | 0,023* | +0,071   | 0,188  | +0,041   | 0,023* |
| LightGBM             | +0,033   | 0,078  | +0,047   | 0,023* | +0,054   | 0,188  | +0,042   | 0,023* |

\* różnica istotna na poziomie  $\alpha = 0,05$  po poprawce Holma. Różnica dodatnia oznacza przewagę LineageDetector (dla Brier score — wartość niższą). Przy 8 parach najniższe osiągalne  $p$  wynosi 0,0078, co ogranicza moc testu.

Wyniki układają się w trzy jednoznaczne wnioski.

**Przewaga nad heurystykami jest istotna i bezdyskusyjna.** We wszystkich dwudziestu porównaniach ( $5 \text{ heurystyk} \times 4 \text{ metryki}$ ) LineageDetector wypada lepiej przy  $p_{\text{Holm}} = 0,039$ , wygrywając na ośmiu z ośmiu grafów (dla reguły korzenia i score’u kompletności — na siedmiu z ośmiu w części metryk). Mediany różnic są przy tym duże: od +0,18 do +0,47 AUC-ROC. Wniosek, że uczenie indukcyjne na cechach topologicznych przewyższa reguły oparte na lokalnym symptomie, ma zatem oparcie statystyczne, a nie tylko w różnicy średnich.

**Przewaga nad metodami boostingowymi jest istotna, ale umiarkowana.** Nad RUSBoostem LineageDetector wygrywa istotnie w AUC-ROC, AUC-PR i Brier ( $p_{\text{Holm}} = 0,023$ , 8/8 gra-

fów), a nad LightGBM — w AUC-PR i Brier (również  $p_{\text{Holm}} = 0,023$ , 8/8). Warto zauważyć, że są to metryki najbardziej wrażliwe na niezrównoważenie klas i na jakość oszacowań prawdopodobieństwa, a więc te, które w zadaniu rankingowym mają największe znaczenie praktyczne.

**Wobec Random Forest przewagi wykazać się nie udało.** Na żadnej z czterech metryk różnica nie osiąga istotności ( $p_{\text{Holm}} = 0,109$ – $0,461$ ), a liczba wygranych grafów (5–6 na 8) jest bliska losowej. Jest to wynik negatywny i jako taki został tu zaraportowany: w świetle zebranych danych LineageDetector i Random Forest należy uznać za *statystycznie nierozróżnialne* pod względem jakości rankingu, a przewagę 0,771 wobec 0,749 AUC-ROC — za mieszczącą się w granicach szumu. Nie oznacza to równoważności obu metod: przy ośmiu parach test wykrywa jedynie efekty duże, więc brak istotności jest równie dobrze zgodny z istnieniem różnicy małej, której przy tej liczbie grafów wykryć się nie da. Rozstrzygnięcie wymagałoby zbioru o większej liczbie grafów.

## 4.5 Badanie ablacyjne detektora

Detektor LineageDetector łączy dwa autorskie składniki nadbudowane nad klasyfikatorem bazowym: relacyjny score kompletności peer-consistency oraz kalibrację izotoniczną. Aby rozdzielić ich udział w wyniku, przeprowadzono badanie ablacyjne w pełnym planie  $2 \times 2$  — każdy ze składników wyłączano osobno i obydwie naraz (parametry `use_completeness` i `calibrate`). Konfiguracja pozostała identyczna z biegiem kanonicznym, dzięki czemu wiersz „pełny detektor” odtwarza dokładnie wartości z tabeli 4.1. Wyniki przedstawia tabela 4.4.

TABELA 4.4: Badanie ablacyjne LineageDetector (protokół indukcyjny, wariant uczciwy, 8 grafów wiarygodnych). W nawiasach — różnica względem pełnego detektora; znak dla Brier score odwrócono tak, aby wartość ujemna zawsze oznaczała pogorszenie.

| Wariant                     | AUC-ROC                  | AUC-PR                   | P@k                | Brier             |
|-----------------------------|--------------------------|--------------------------|--------------------|-------------------|
| Pełny detektor              | 0,771                    | 0,435                    | <b>0,443</b>       | <b>0,106</b>      |
| bez kalibracji izotonicznej | <b>0,786</b><br>(+0,015) | <b>0,440</b><br>(+0,005) | 0,446<br>(+0,003)  | 0,111<br>(−0,005) |
| bez score’u kompletności    | 0,747<br>(−0,023)        | 0,420<br>(−0,015)        | 0,401<br>(−0,042)* | 0,107<br>(−0,001) |
| bez obu składników          | 0,750<br>(−0,020)        | 0,425<br>(−0,010)        | 0,387<br>(−0,056)  | 0,116<br>(−0,010) |

\* ubytek istotny statystycznie (test Wilcoxona sparowany po grafach, poprawka Holma w obrębie metryki,  $p_{\text{Holm}} = 0,023$ , pogorszenie na 8 z 8 grafów). Pozostałe różnice nie osiągają istotności.

**Score kompletności odpowiada za wkład detektora.** Jego wyłączenie pogarsza wszystkie cztery metryki, a w przypadku Precision@k — najbardziej praktycznej z nich, bo opisującej trafność czołówki rankingu przekazywanej do ręcznej inspekcji — ubytek  $-0,042$  jest istotny statystycznie i konsekwentny: wariant bez kompletności przegrywa na *wszystkich* ośmiu grafach. Bez tego składnika detektor spada do poziomu Random Forest (0,747 wobec 0,749 AUC-ROC), co jest spójne z jego konstrukcją — bazowym estymatorem jest właśnie Random Forest, więc po usunięciu cech relacyjnych oba modele stają się niemal tożsame. Sygnał peer-consistency jest zatem tym elementem, który odróżnia autorski detektor od gotowego klasyfikatora.

**Kalibracja izotoniczna nie poprawia rankingu — pogarsza go.** Wynik ten jest sprzeczny z oczekiwaniem i wymaga jasnego zaraportowania: wariant *bez* kalibracji osiąga wyższe AUC-ROC (0,786 wobec 0,771) oraz nieznacznie wyższe AUC-PR i Precision@k, wygrywając odpowiednio na 7 i 5 z 8 grafów. Kalibracja poprawia natomiast to, do czego służy — Brier score (0,106 wobec 0,111, poprawa na 6 z 8 grafów) — choć i ta różnica nie osiąga istotności statystycznej. Mechanizm jest znany: regresja izotoniczna jest funkcją schodkową, więc przypisuje identyczne

prawdopodobieństwo całym grupom węzłów, wprowadzając do rankingu remisy, których surowy klasyfikator nie miał; dodatkowo w schemacie `CalibratedClassifierCV` z  $cv = 3$  każdy model składowy uczy się na dwóch trzecich puli treningowej. Kalibracja realizuje więc *wymianę*: oddaje część zdolności rozdzielniczej w zamian za rzetelność oszacowań prawdopodobieństwa.

**Wniosek dla konstrukcji detektora.** Uzasadnione empirycznie jest zachowanie kalibracji tylko wtedy, gdy odbiorcą wyniku ma być wartość  $p$ (zainfekowana) interpretowana jako prawdopodobieństwo — na przykład przy progowaniu ryzyka albo szacowaniu kosztu inspekcji. Jeśli celem jest wyłącznie uszeregowanie tabel do przeglądu, wariant bez kalibracji jest lepszym wyborem. W pracy jako konfigurację główną zachowano wariant skalibrowany, ponieważ zadanie zdefiniowano (sekcja 3.1) jako oszacowanie prawdopodobieństwa, a nie sam ranking; alternatywa jest jednak udokumentowana powyżej i pozostaje dostępna jako parametr implementacji.

## 4.6 Kontrola przecieku i analiza wrażliwości

### 4.6.1 Przeciek izolacji

Usunięcie zadania może pozbawić tabelę *wszystkich* krawędzi przepływu danych, czyniąc ją w pełni izolowaną w podgrafie `DATA_FLOW`. Ponieważ w oryginalnych grafach tabele izolowane nie występują, trywialna reguła „tabela izolowana  $\Rightarrow$  zainfekowana” pozwala rozpoznać znaczną część pozytywów bez żadnego uczenia i sztucznie zawyża wynik. Aby zmierzyć skalę tego zjawiska, każdą metodę oceniono w dwóch wariantach: *napompowanym* (izolowane pozostają w puli kandydatów) oraz *uczciwym* (flaga `exclude_isolated` usuwa je z puli). Tabela 4.5 zestawia oba warianty.

TABELA 4.5: Wpływ przecieku izolacji na AUC-ROC (protokół indukcyjny).

| Metoda               | Napompowany  | Uczciwy      |
|----------------------|--------------|--------------|
| Brzeg (root/leaf)    | 0,600        | 0,455        |
| Niska łączność zadań | 0,518        | 0,324        |
| Random Forest        | 0,816        | 0,749        |
| LightGBM             | 0,805        | 0,720        |
| LineageDetector*     | <b>0,842</b> | <b>0,771</b> |

Heurystyki lokalne tracą najwięcej: `boundary` spada z 0,600 do 0,455, a `low_job_connectivity` z 0,518 do 0,324 — „żyły”one głównie z przecieku izolacji. Metody uczone spadają znacznie łagodniej (RF: 0,816 $\rightarrow$ 0,749; LineageDetector: 0,842 $\rightarrow$ 0,771), co pokazuje, że opierają się na rzeczywistym sygnale strukturalnym, a nie na trywialnej regule. W obu wariantach LineageDetector pozostaje najlepszy.

Oba warianty uśredniono po *różnej* liczbie grafów wiarygodnych: uczciwy — 8, napompowany — 10. Wykluczenie izolowanych zmniejsza liczbę pozytywów, przez co dwa grafy spadają poniżej progu `MIN_POS = 5`. Z tego powodu wartości w tabeli 4.5 ilustrują tendencję (metody uczone są odporniejsze na usunięcie przecieku), a nie dokładny ubytek — nie należy odejmować ich kolumn 1:1.

### 4.6.2 Wrażliwość na odsetek usuwanych zadań

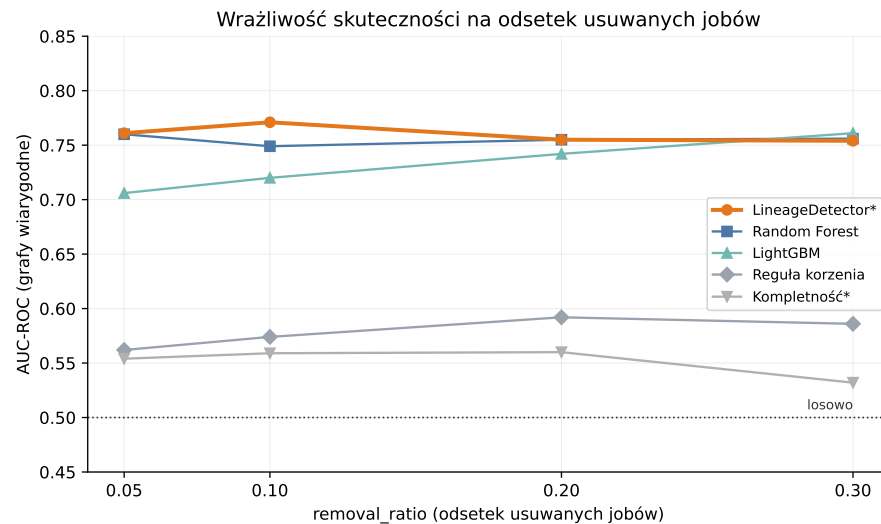
Zbadano wpływ parametru `removal_ratio` (udział usuwanych zadań łączących) na wynik LineageDetector, przy wartościach 0,05 / 0,10 / 0,20 / 0,30 (tabela 4.6).

Wynik jest stabilny: w całym badanym zakresie AUC-ROC mieści się w przedziale 0,754–0,771, a różnice są rzędu wariancji między powtórzeniami. Oznacza to, że skuteczność detektora nie za-

TABELA 4.6: Wrażliwość LineageDetectora na `removal_ratio` (AUC-ROC, protokół indukcyjny).

| <code>removal_ratio</code> | 0,05  | 0,10  | 0,20  | 0,30  |
|----------------------------|-------|-------|-------|-------|
| AUC-ROC                    | 0,761 | 0,771 | 0,755 | 0,754 |

leży istotnie od intensywności symulowanego broken lineage — zarówno przy rzadkich (5%), jak i częstych (30%) usunięciach zadań detektor zachowuje porównywalną jakość rankingu (rys. 4.2).



\* wkład własny. Wynik LineageDetectora niemal stały (0,754–0,771) — nie jest artefaktem wyboru `removal_ratio`.

RYSUNEK 4.2: Wrażliwość AUC-ROC detektora LineageDetector na parametr `removal_ratio`. Wynik pozostaje stabilny w całym badanym zakresie.

## 4.7 Ważność cech

Badanie ablacyjne pokazało, że score kompletności wnosi informację; analiza ważności cech odpowiada na pytanie, *które* elementy opisu węzła niosą sygnał. Zastosowano metodę permutacyjną: wartości pojedynczej cechy przestawia się losowo w zbiorze testowym, a spadek AUC-ROC mierzy, ile model tracił bez tej informacji. W odróżnieniu od ważności wbudowanej w drzewa decyzyjne (opartej na liczbie podziałów) metoda ta nie faworyzuje cech o wielu unikatowych wartościach i mierzy wpływ na metrykę, która faktycznie podlega ocenie. Wyniki uśredniono po 10 permutacjach, ośmiu grafach wiarygodnych i trzech ziarnach (tabela 4.7).

**Najsilniejszy sygnał pochodzi z otoczenia, nie z samego węzła.** Dwie najważniejsze cechy — średni stopień wejściowy sąsiednich zadań oraz liczba sąsiadujących zadań — opisują nie tabelę, lecz jej kontekst. Wyprzedzają one cechy opisujące bezpośrednio symptom zerwania (`in_df`, `is_root`), które plasują się w dolnej połowie zestawienia. Potwierdza to diagnozę z sekcji 4.10: sam brak producenta nie odróżnia tabeli osieroconej od prawdziwego źródła, a informacja rozstrzygająca leży w sąsiedztwie węzła.

**Wkład autorskiego score’u jest widoczny, lecz jednostronny.** Cecha `peer_producer_score` zajmuje piąte miejsce na szesnaście i jest najwyższą sklasyfikowaną cechą relacyjną; `completeness_anomaly` plasuje się ósma. Zwraca jednak uwagę `peer_consumer_score`: jej przestawienie nie zmienia wyniku *w ogóle* — spadek wynosi dokładnie zero przy zerowym odchyleniu standardowym we wszystkich 24 kombinacjach grafu i ziarna.

TABELA 4.7: Ważność cech detektora LineageDetector mierzona metodą permutacyjną: średni spadek AUC-ROC po losowym przestawieniu wartości cechy. Gwiazdka — cecha wnoszona przez autorski score kompletności.

| Cecha                                         | Spadek AUC-ROC | Odch. std |
|-----------------------------------------------|----------------|-----------|
| avg_job_in_df — śr. stopień wej. zadań        | +0,0616        | 0,0510    |
| n_job_neighbors — liczba sąsiednich zadań     | +0,0492        | 0,0266    |
| out_df — wyjściowe krawędzie <i>DATA_FLOW</i> | +0,0408        | 0,0207    |
| degree — stopień całkowity                    | +0,0260        | 0,0336    |
| peer_producer_score*                          | +0,0238        | 0,0240    |
| n_field_children — liczba pól tabeli          | +0,0196        | 0,0280    |
| role — rola w przepływie                      | +0,0128        | 0,0204    |
| completeness_anomaly*                         | +0,0107        | 0,0140    |
| is_root — brak producenta                     | +0,0100        | 0,0110    |
| flow_depth — głębokość w DAG                  | +0,0072        | 0,0130    |
| in_df — wejściowe krawędzie <i>DATA_FLOW</i>  | +0,0065        | 0,0125    |
| is_leaf — brak konsumenta                     | +0,0036        | 0,0128    |
| depth_plus_reach — długość ścieżki            | +0,0018        | 0,0292    |
| peer_consumer_score*                          | 0,0000         | 0,0000    |
| avg_job_out_df — śr. stopień wyj. zadań       | −0,0008        | 0,0046    |
| flow_reach — zasięg w dół DAG                 | −0,0015        | 0,0276    |

Analiza danych wyjaśnia przyczynę i jest ona strukturalna, nie implementacyjna. W zbiorze DLG-DG-23 *każde* zadanie zapisuje dokładnie jedną tabelę wyjściową — sprawdzono to na wszystkich zadaniach we wszystkich grafach. Zbiór peerów po stronie konsumenta  $C(t)$ , zdefiniowany jako inne tabele zapisywane przez te same zadania upstream, jest zatem zawsze pusty, a wynikający z niego score — tożsamościowo równy zeru. Konsekwencje są dwie. Po pierwsze, definicja sygnału anomalii z sekcji 3.5.3 redukuje się na tym zbiorze do członu producenta:  $\text{anomaly}(t) = 1[\text{in\_df}(t) = 0] \cdot \text{prod}(t)$ . Po drugie, detektor korzysta w praktyce z 15, a nie 16 cech.

Nie jest to błąd konstrukcji — sformułowanie score’u pozostaje symetryczne i na zbiorze dopuszczającym zadania o wielu wyjściach (np. operacje partycjonowania lub rozgałęzienia potoku) strona konsumenta byłaby aktywna. Obserwacja ta ma jednak dwie praktyczne implikacje. Wskazuje, że potencjał score’u peer-consistency jest na zbiorze DLG-DG-23 wykorzystany tylko w połowie, co może częściowo tłumaczyć umiarkowaną skalę przewagi detektora (sekcja 4.4). Wzmacnia też wyjaśnienie porażki sygnału relacyjnego na zewnętrznym grafie DUT (sekcja 4.9): tam nieaktywna jest również strona producenta, przez co score milknie całkowicie.

## 4.8 Warianty protokołu ewaluacji

Wyniki główne uzyskano w jednym, ściśle określonym protokole: indukcyjnym leave-one-graph-out, z losowym usuwaniem 10% zadań łączących. Powstaje pytanie, na ile wnioski zależą od tego wyboru. Zbadano zatem trzy warianty kontrolne, zmieniające po jednym elemencie protokołu przy zachowaniu pozostałych ustawień biegu kanonicznego. Tabela 4.8 zestawia wyniki; dla porównywalności wszystkie warianty ograniczono do tych samych pięciu dużych grafów, na których każdy z nich daje wiarygodne metryki.

### 4.8.1 Protokół transduktywny

W wariancie transduktywnym podział na zbiór treningowy i testowy przebiega *wewnątrz* grafu (60/40 węzłów), a nie między grafami. Model widzi zatem w treningu fragment tej samej struktury, na której jest oceniany. Wszystkie metody uczone zyskują, co potwierdza, że protokół indukcyjny przyjęty jako główny jest istotnie trudniejszy — a więc uczciwszy.

TABELA 4.8: Wpływ zmiany protokołu ewaluacji na AUC-ROC. Wszystkie warianty na tych samych pięciu grafach testowych (DLG6, DLG10, DLG12, DLG16, DLG18), `removal_ratio = 0,10`, wariant uczciwy.

| Metoda               | Indukcyjny<br>(kanoniczny) | Transduktywny | Skala: małe→duże |
|----------------------|----------------------------|---------------|------------------|
| Anomalia stopnia     | 0,410                      | 0,392         | 0,410            |
| Brzeg (root/leaf)    | 0,480                      | 0,482         | 0,480            |
| Niska łączność zadań | 0,355                      | 0,373         | 0,355            |
| Random Forest        | 0,774                      | <b>0,819</b>  | 0,626            |
| RUSBoost             | 0,602                      | 0,634         | 0,613            |
| LightGBM             | 0,731                      | 0,769         | <b>0,638</b>     |
| LineageDetector*     | <b>0,785</b>               | 0,804         | 0,623            |

Rozkład tego zysku jest jednak nierówny i sam w sobie stanowi wynik. Random Forest zyskuje najwięcej (+0,045: z 0,774 do 0,819), LineageDetector wyraźnie mniej (+0,019: z 0,785 do 0,804). Innymi słowy: przewaga Random Forest ujawnia się dopiero wtedy, gdy model może dopasować się do konkretnego grafu. Sygnał relacyjny wykorzystywany przez LineageDetector przenosi się natomiast między grafami lepiej, co jest zgodne z jego konstrukcją — porównanie tabeli z jej peerami jest z natury niezależne od globalnych właściwości grafu. W protokole transduktywnym Random Forest wysuwa się na prowadzenie, w indukcyjnym obie metody są nierozróżnialne (sekcja 4.4).

#### 4.8.2 Generalizacja na większą skalę

Wariant trzeci sprawdza najtrudniejszy przypadek praktyczny: model uczony na 10 małych i 3 średnich grafach ocenia grafy duże, o rząd wielkości liczniejsze w węzły. Skuteczność wszystkich metod uczonych spada drastycznie — z okolic 0,73–0,79 do 0,61–0,64 AUC-ROC, czyli o około 0,15. Heurystyki, nie korzystając z treningu, pozostają na niezmiennym poziomie.

Jest to najpoważniejsze ograniczenie praktyczne ujawnione w pracy: wzorce topologiczne wyuczone na małych grafach nie przenoszą się w pełni na grafy duże. W protokole kanonicznym efekt ten nie występuje, ponieważ pula treningowa *leave-one-graph-out* zawiera zawsze inne grafy duże. Wniosek dla zastosowań: detektor należy trenować na grafach o skali zbliżonej do docelowej, a zbiór treningowy złożony wyłącznie z małych instancji nie wystarcza. LineageDetector nie ma w tym wariantcie przewagi (0,623 wobec 0,638 dla LightGBM) — sygnał peer-consistency, kalibrowany na małych grafach o innej gęstości połączeń, traci trafność.

#### 4.8.3 Pojedyncze zdarzenie broken lineage

Wariant kontrolny o najczystszej interpretacji przyczynowej: zamiast usuwać 10% zadań naraz, każda instancja testowa powstaje przez usunięcie *dokładnie jednego* zadania (do 30 prób na graf, metryki uśredniane). Jedno zdarzenie broken lineage odpowiada tu jednemu brakującemu zadaniu, a graf pozostaje poza tym nienaruszony. Trening prowadzony jest nadal na puli wielozadaniowej, na grafach niewidzianych. Wyniki zestawia tabela 4.9.

Wynik rozchodzi się w dwie przeciwne strony i oba kierunki są pouczające. **AUC-ROC rośnie** — dla LineageDetectora z 0,771 do 0,799, dla Random Forest z 0,749 do 0,792 — ponieważ przy jednym usuniętym zadaniu reszta grafu pozostaje spójna, a nieliczne tabele osieroczone wyraźniej odcinają się od tła. **AUC-PR spada natomiast ponaddwukrotnie** (0,435 → 0,192 dla detektora), gdyż klasa pozytywna liczy teraz średnio dwie–trzy tabele na 260 kandydatów, czyli poniżej jednego procenta.

TABELA 4.9: Wariant pojedynczego usuniętego zadania w zestawieniu z biegiem kanonicznym (protokół indukcyjny, wariant uczciwy, 8 grafów wiarygodnych).

| Metoda             | AUC-ROC       |              | AUC-PR        |              |
|--------------------|---------------|--------------|---------------|--------------|
|                    | jedno zadanie | kanoniczny   | jedno zadanie | kanoniczny   |
| Reguła korzenia    | 0,518         | 0,574        | 0,025         | 0,168        |
| Score kompletności | 0,546         | 0,559        | 0,060         | 0,206        |
| Random Forest      | 0,792         | 0,749        | 0,208         | 0,425        |
| RUSBoost           | 0,682         | 0,631        | 0,071         | 0,249        |
| LightGBM           | 0,767         | 0,720        | 0,187         | 0,382        |
| LineageDetector*   | <b>0,799</b>  | <b>0,771</b> | 0,192         | <b>0,435</b> |

Rozbieżność ta jest praktycznie istotna i stanowi argument za raportowaniem obu metryk. Wysokie AUC-ROC oznacza, że model niemal zawsze umieszcza tabelę zainfekowaną powyżej losowo wybranej tabeli czystej. Niskie AUC-PR mówi jednak, że w czołówce rankingu przekazywanej do ręcznej inspekcji trafienia i tak będą rzadkie — bo pozytywów jest garstka. Jest to wierne odwzorowanie sytuacji praktycznej: w dużym grafie produkcyjnym z pojedynczym zerwaniem lineage znalezienie właściwej tabeli pozostaje trudne, mimo dobrego uporządkowania rankingu.

Uporządkowanie metod nie zmienia się: LineageDetector osiąga najwyższe AUC-ROC i zdecydowanie najlepszy Brier score (0,042 wobec 0,059 dla Random Forest i 0,117 dla LightGBM), a heurystyki pozostają w pobliżu poziomu losowego. Wniosek jakościowy z biegu kanonicznego jest zatem odporny również na tę zmianę definicji instancji testowej.

## 4.9 Test na grafie spoza zbioru DLG-DG-23 (cross-dataset)

Dotychczasowe eksperymenty oceniały metody w protokole indukcyjnym *wewnątrz* jednego zbioru danych. Aby sprawdzić przenośność metody na dane z zupełnie innego źródła, przeprowadzono test cross-dataset: model wytrenowano wyłącznie na grafach DLG-DG-23 (małych i średnich, zgodnie ze schematem generalizacji do skali z sekcji 4.6), a następnie oceniono — bez docuczenia — na zewnętrznym grafie *DUT* zbudowanym z repozytorium grupy promotora [15]. Graf DUT pochodzi z bazy *Northwind* i syntetycznych scenariuszy SQL; jego lineage zapisany jest na poziomie wierszy (RDF/CSV), dlatego opracowano adapter agregujący go do modelu tabela-zadanie-*DATA\_FLOW* zgodnego z DLG-DG-23: dla każdej tabeli docelowej tworzy się węzeł-zadanie karmiony przez jej tabele źródłowe. Powstały graf liczy 21 tabel, 14 zadań i 33 krawędzie przepływu — jest więc o rzędy wielkości mniejszy i ma inną, łańcuchową topologię.

Tabela 4.10 przedstawia wyniki na grafie DUT w wariancie uczciwym (z wykluczeniem węzłów izolowanych), uśrednione po 20 powtórzeniach.

Najważniejsza obserwacja: metody uczone *nie załamują się* na grafie z innego źródła. Random Forest (0,625), LightGBM (0,674) i LineageDetector (0,641) osiągają na DUT AUC-ROC porównywalne z ich wynikiem na dużych grafach DLG-DG-23 (odpowiednio 0,617, 0,605, 0,623) — mimo że graf testowy pochodzi z zupełnie innej dziedziny i jest znacznie mniejszy. LineageDetector zachowuje przy tym najlepszą kalibrację (Brier 0,187). W wariancie napompowanym (bez wykluczania izolowanych) przewaga jest jeszcze wyraźniejsza — LineageDetector osiąga AUC-ROC 0,773 i AUC-PR 0,721. Potwierdza to, że wzorce topologiczne wyuczone na jednym zbiorze niosą użyteczny sygnał także poza nim.

Wynik wymaga jednak trzech zastrzeżeń. Po pierwsze, graf DUT jest bardzo mały (21 tabel), więc liczba zainfekowanych tabel spada poniżej progu wiarygodności przyjętego dla DLG-DG-23; test należy traktować jako jakościowy dowód przenośności (proof of concept), a nie precyzyjny



TABELA 4.10: Wynik na zewnętrznym grafie DUT (trening wyłącznie na DLG-DG-23, wariant uczciwy). Ostatnia kolumna — AUC-ROC na dużych grafach DLG-DG-23 jako odniesienie wewnątrz zbioru.

| Metoda               | Graf zewnętrzny DUT |        |       |              | duże DLG |
|----------------------|---------------------|--------|-------|--------------|----------|
|                      | AUC-ROC             | AUC-PR | P@k   | Brier        | AUC-ROC  |
| Anomalia stopnia     | 0,751               | 0,422  | 0,436 | 0,180        | 0,426    |
| Niska łączność zadań | 0,751               | 0,422  | 0,436 | 0,198        | 0,384    |
| Score kompletności   | 0,500               | 0,262  | 0,189 | 0,262        | 0,568    |
| Random Forest        | 0,625               | 0,497  | 0,380 | 0,225        | 0,617    |
| RUSBoost             | 0,489               | 0,260  | 0,141 | 0,216        | 0,591    |
| LightGBM             | 0,674               | 0,564  | 0,464 | 0,296        | 0,605    |
| LineageDetector*     | 0,641               | 0,521  | 0,439 | <b>0,187</b> | 0,623    |

benchmark. Po drugie, na tym konkretnym grafie trywialne heurystyki stopnia osiągają wysokie AUC-ROC (0,751) — odwrotnie niż wewnątrz DLG-DG-23, gdzie plasują się w okolicy poziomu losowego. Wynika to z łańcuchowej topologii DUT: po usunięciu zadania osierocona tabela ma tu wyraźnie niższy stopień, więc sam stopień jest na tak prostej strukturze silnym sygnałem — który jednak nie generalizuje na gęstsze grafy produkcyjne. Po trzecie, relacyjny score kompletności załamuje się na DUT (AUC-ROC 0,500), ponieważ łańcuchowe scenariusze dają każdej tabeli co najwyżej jednego peera — sygnał peer-consistency wymaga rozgałęzień, których w tym grafie brak. Stąd na DUT LineageDetector nie zyskuje przewagi nad czystym LightGBM, choć pozostaje najlepiej skalibrowany.

Aby sprawdzić, czy powyższe obserwacje nie są artefaktem jednej konfiguracji, powtórzono test w przeglądzie obejmującym dwa warianty izolacji oraz trzy wartości `removal_ratio` (0,2 / 0,3 / 0,4), po 25 powtórzeniach na konfigurację. Przegląd potwierdza wszystkie trzy zastrzeżenia i dodaje dwie obserwacje. Po pierwsze, jakość metod uczonych rośnie monotonicznie wraz z natężeniem symulowanego zjawiska (w wariancie napompowanym AUC-ROC LineageDetector wynosi kolejno 0,767, 0,812 i 0,843) — modele reagują zatem na intensywność broken lineage, a nie na szum. Po drugie, score kompletności osiąga AUC-ROC *dokładnie* 0,500 we wszystkich sześciu konfiguracjach; nie jest to więc słaby wynik losowy, lecz sygnał tożsamościowo zerowy, wynikający ze struktury grafu: w topologii łańcuchowej tabela nie ma peerów, z którymi mogłaby zostać porównana. Odnotować należy również, że przy najsilniejszym zaburzeniu (`removal_ratio` = 0,4), gdy liczba tabel zainfekowanych zbliża się do progu wiarygodności, LineageDetector wraca na prowadzenie także na DUT (AUC-ROC 0,684 wobec 0,656 dla LightGBM, przy najwyższym P@k 0,555) — przewaga detektora ujawnia się więc dopiero przy dostatecznej liczbie pozytywów, spójnie z rolą liczby pozytywów obserwowaną w sekcji 4.3. Jako liczby główne zachowano wariant `removal_ratio` = 0,3, mieszczący się w zakresie zbadanym w analizie wrażliwości (sekcja 4.6); pełne zestawienie przeglądu zawiera plik `results/cross_dataset_dut_sweep.md` w repozytorium.

Pełne, ilościowe porównanie na drugim zbiorze wymagałoby większego grafu zewnętrznego o topologii rozgałęzionej — kierunek ten wskazano w rozdziale 5.

## 4.10 Analiza wyników

**Skąd bierze się wynik LineageDetector.** Zestawienie sekcji 4.4 i 4.5 pozwala przypisać wynik konkretnym składnikom, zamiast tłumaczyć go samą kombinacją trzech źródeł sygnału. Cechy pozycji w DAG opisują węzeł lokalnie, lecz nie odróżniają osieroconej tabeli-następnika od prawdziwego źródła. Sygnał relacyjny wnosi dopiero score kompletności *peer-consistency* — porównanie

tabeli z jej peerami wpadającymi do tych samych zadań — i to on odpowiada za całą przewagę detektora nad klasyfikatorem bazowym: po jego wyłączeniu detektor spada do poziomu Random Forest, a ubytek Precision@k jest istotny statystycznie. Kalibracja izotoniczna działa natomiast inaczej, niż zakładano przy projektowaniu detektora: nie jest neutralna dla rankingu, lecz go nieznacznie pogarsza, poprawiając w zamian rzetelność oszacowań prawdopodobieństwa (Brier 0,106 wobec 0,111 bez kalibracji). Jest to zatem świadoma wymiana jakości rankingu na jakość prawdopodobieństw, a nie ulepszenie bezkosztowe.

**Czego wynik nie dowodzi.** Przewaga nad heurystykami oraz nad metodami boostingowymi ma potwierdzenie statystyczne, natomiast różnica wobec Random Forest — nie. Twierdzenie „autorski detektor jest najlepszą z badanych metod” byłoby więc nadinterpretacją średnich z tabeli 4.1. Uprawniony wniosek jest węższy: detektor dorównuje najlepszemu klasycznemu klasyfikatorowi przy istotnie lepszej trafności czołówki rankingu niż metody boostingowe, a jego przewaga nad nimi pochodzi z jednego, dającego się wskazać składnika.

**Dlaczego heurystyki oparte na wspólnych sąsiadach zawodzą.** Podgraf przepływu danych jest *dwudzielny* (tabele łączą się wyłącznie z zadaniami, nie bezpośrednio z innymi tabelami), więc miary typu Common Neighbors, Jaccard czy Adamic-Adar są tożsamościowo zerowe między węzłami tego samego typu i nie niosą informacji — dlatego wykluczono je z implementacji. Heurystyki oparte na stopniu i roli w grafie mierzą jedynie lokalny symptom, który po usunięciu przecieku izolacji przestaje wystarczać.

**Główna trudność zadania.** Najtrudniejszym przypadkiem pozostaje osierocona tabela-następnik: po usunięciu zadania traci ona producenta ( $\text{in\_df} = 0$ ) i staje się strukturalnie identyczna z prawdziwą tabelą źródłową. Rozróżnienie ich jest możliwe tylko dzięki kontekstowi relacyjnemu (czy peerzy również są źródłami) oraz uczeniu na innych grafach — co wyjaśnia, dlaczego metody indukcyjne przewyższają heurystyki, a sygnał topologiczny, choć użyteczny, ma naturalny sufit skuteczności ( $\text{AUC-ROC} \approx 0,77$ ) bez dostępu do semantyki nazw i schematów.

## 4.11 Zagrożenia trafności

Wnioski z poprzednich sekcji obarczone są ograniczeniami wynikającymi z konstrukcji eksperymentu i z właściwości dostępnych danych. Omówiono je poniżej w czterech kategoriach przyjmowanych w badaniach empirycznych w inżynierii oprogramowania [41]: trafności konstrukcji, wewnętrznej, zewnętrznej oraz wnioskowania statystycznego.

### 4.11.1 Trafność konstrukcji

**Etykieta zastępcza zamiast obserwowanego zjawiska.** Za tabelę zainfekowaną uznaje się każdą tabelę incydentną do usuniętego zadania. Jest to definicja operacyjna, nie pomiar rzeczywistego zerwania lineage: w praktyce część takich tabel mogłaby zachować czytelne pochodzenie inną ścieżką (np. przez równoległe zadanie zapisujące te same dane), a inne tabele mogłyby być dotknięte zerwaniem, którego symulacja nie obejmuje. Miara skuteczności odnosi się zatem do zadania zdefiniowanego w pracy, a nie bezpośrednio do użyteczności dla audytora.

**Utożsamienie różnych przyczyn broken lineage.** Tabela tymczasowa, funkcja definiowana przez użytkownika i zapytanie rekurencyjne są modelowane jednakowo — jako usunięcie węzła-zadania. Upraszcza to model i czyni go niezależnym od mechanizmu, lecz uniemożliwia sprawdzenie, czy detektor radzi sobie równie dobrze z każdą z tych przyczyn osobno.

**Zbieżność metryk przy przyjętym  $k$ .** W biegu kanonicznym  $k$  równa się liczbie tabel faktycznie zainfekowanych, przez co Precision@k, Recall@k i Hits@k przyjmują tę samą wartość. Metryki

te nie są więc niezależnymi świadectwami jakości, a ich zgodność nie stanowi dodatkowego potwierdzenia wyniku.

#### 4.11.2 Trafność wewnętrzną

**Przeciek izolacji.** Usunięcie zadania może pozostawić tabelę bez żadnej krawędzi przepływu, co czyni ją rozpoznawalną trywialną regułą. Zjawisko to zidentyfikowano i zmierzono (tabela 3.2: 19–35% pozytywów), a główne wyniki raportowane są w wariancie z wykluczeniem takich tabel. Jest to zabezpieczenie, nie eliminacja problemu: pozostają tabele o częściowo zerwanym lineage, których łatwość wykrycia również zależy od liczby utraconych krawędzi.

**Losowy dobór usuwanych zadań.** Zadania usuwane są losowo, podczas gdy w praktyce zerwanie lineage nie jest zdarzeniem losowym — dotyczy częściej pewnych klas obiektów (potoków wsadowych, procesów tymczasowych). Rozkład symulowanych uszkodzeń może być zatem łatwiejszy albo trudniejszy od rzeczywistego; kierunek tego obciążenia pozostaje nieznany.

**Brak strojenia hiperparametrów.** Wszystkie klasyfikatory uruchomiono w konfiguracjach domyślnych lub zbliżonych do domyślnych, bez przeszukiwania przestrzeni hiperparametrów. Zapewnia to jednakowe traktowanie metod i eliminuje ryzyko przestrojenia pod zbiór testowy, lecz oznacza, że porównanie dotyczy metod „z pudełka”, a nie ich potencjału maksymalnego.

#### 4.11.3 Trafność zewnętrzną

**Jeden zbiór danych, jedna domena.** Wyniki główne pochodzą z 18 grafów jednego dostawcy chmurowego. Nie wiadomo, na ile wzorce topologiczne charakterystyczne dla tego środowiska przenoszą się na inne architektury przetwarzania danych. Test cross-dataset (sekcja 4.9) częściowo tę lukę adresuje, lecz graf zewnętrzny jest mały (21 tabel) i ma odmienną, łańcuchową topologię — stanowi więc dowód jakościowy, a nie ilościowe potwierdzenie przenośności.

**Ograniczenie do topologii jest wyborem, nie tylko koniecznością.** Anonimizacja zbioru wymusiła rezygnację z sygnału semantycznego, co jest zarazem przedmiotem pytania badawczego. Uzyskany sufit skuteczności ( $AUC-ROC \approx 0,77$ ) opisuje jednak wyłącznie podejście czysto topologiczne i nie stanowi granicy dla metod korzystających z nazw obiektów lub metadanych schematu.

#### 4.11.4 Trafność wnioskowania statystycznego

**Mała liczba jednostek obserwacji.** Testy sparowano po ośmiu grafach wiarygodnych, co ogranicza moc: najniższe osiągalne  $p$  w dokładnym teście Wilcoxona wynosi 0,0078, a efekty małe pozostają niewykrywalne. Brak istotności różnicy wobec Random Forest (sekcja 4.4) należy zatem czytać jako *brak dowodu na różnicę*, a nie jako dowód jej braku.

**Zależność między powtórzeniami.** Trzy powtórzenia biegu kanonicznego różnią się wyłącznie ziarnem losowości doboru usuwanych zadań; operują na tych samych grafach, więc nie zwiększają liczby niezależnych jednostek obserwacji. Uśrednianie po ziarnach (z trzymowaniem wartości skrajnych) zmniejsza wariancję oszacowań, lecz nie zwiększa mocy testu.

**Wybór progu wiarygodności.** Próg  $MIN\_POS = 5$  przyjęto arbitralnie. Decyduje on o tym, które grafy wchodzi do agregacji, a przy ośmiu jednostkach obserwacji pojedynczy graf ma zauważalny wpływ na średnią — dlatego obok średnich raportowane są wyniki w rozbiciu na grafy (tabela 4.2).

## Rozdział 5

# Uwagi końcowe

Rozdział podsumowuje uzyskane wyniki, odnosząc je do postawionych pytań badawczych, oraz wskazuje kierunki dalszych badań.

### 5.1 Podsumowanie

Celem pracy było zbadanie, w jakim stopniu sama topologia grafu lineage pozwala wskazać tabele o zerwanym pochodzeniu danych. Poniżej odniesiono się do postawionych pytań badawczych.

**PB1 — Techniki ekstrakcji grafów lineage.** Praca korzysta z gotowych, rzeczywistych grafów zbioru DLG-DG-23, wczytywanych z reprezentacji JSON (węzły + krawędzie z typami `asset_type`, `relation_type`) do postaci skierowanego grafu DAG przepływu danych. Pokazano, że dla zadania detekcji węzłów wystarcza podgraf przepływu (krawędzie `DATA_FLOW` między tabelami a zadaniami), bez potrzeby odtwarzania semantyki nazw czy schematów.

**PB2 — Adaptacja ML do wykrywania niekompletności lineage.** Zaadaptowano trzy klasyczne klasyfikatory (Random Forest, RUSBoost, LightGBM) z zadania klasyfikacji par obiektów do klasyfikacji *węzła* na podstawie 13-elementowego wektora cech topologicznych, oraz zaproponowano autorski detektor LineageDetector. Wykazano — z potwierdzeniem statystycznym — że metody uczone przewyższają heurystyki, a LineageDetector przewyższa metody boostingowe (RUSBoost, LightGBM), osiągając najwyższe wartości wszystkich metryk: AUC-ROC 0,771 przy Brier score 0,106. Wobec Random Forest przewagi wykazać się nie udało — obie metody należy uznać za statystycznie nierozróżnialne przy dostępnej liczbie ośmiu grafów testowych. Badanie ablacyjne przypisało przewagę detektora relacyjnemu score’owi kompletności: po jego usunięciu wynik spada do poziomu klasyfikatora bazowego.

**PB3 — Obsługa rekurencji (CTE) i UDF.** W przyjętym ujęciu przyczyny broken lineage (UDF, tabele tymczasowe, zapytania rekurencyjne) są modelowane jednolicie — jako usunięcie węzła-zadania wraz z jego krawędziami — niezależnie od konkretnego mechanizmu. Dzięki temu detektor nie wymaga osobnej obsługi każdego typu transformacji; ograniczeniem jest brak rozróżnienia przyczyny zerwania, co pozostaje otwartym kierunkiem.

**PB4 — Metryki i datasety.** Zastosowano zbiór DLG-DG-23 jako *ground truth* oraz metryki rankingowe czułe na niezrównoważenie klas (AUC-ROC, AUC-PR, Precision@k/Recall@k/Hits@k, Brier). Zaproponowano protokół ewaluacji *indukcyjnej* cross-graph oraz kontrolę przecieku izolacji (`exclude_isolated`) — element metodyczny niezbędny dla uczciwej oceny, dotąd nieobecny

w literaturze broken lineage. Porównania metod oparto na teście kolejności par Wilcozona sparowanym po grafach, zgodnie z zaleceniem Demšara [13]; ujawniło ono ograniczenie samego zbioru: przy ośmiu grafach wiarygodnych najniższe osiągalne  $p$  wynosi 0,0078, przez co efekty małe pozostają niewykrywalne niezależnie od jakości metody. Jest to argument za rozbudową benchmarków lineage o większą liczbę grafów, a nie wyłącznie o grafy większe.

**Wniosek główny.** Sama topologia grafu lineage niesie umiarkowany, lecz użyteczny sygnał o broken lineage na poziomie węzłów (LineageDetector, AUC-ROC 0,771 w wariancie uczciwym i indukcyjnym). Praca pozycjonuje się jako *topology-only baseline* — punkt odniesienia dla bogatszych metod grafowo-semantycznych (grafy wiedzy, GNN), pokazujący, ile można osiągnąć bez dostępu do nazw i schematów, oraz uzupełniający dotychczasowe podejścia o brakujące ujęcie węzłowe: wskazanie podejrzanych tabel może poprzedzać kosztowniejsze odtwarzanie konkretnych zależności.

## 5.2 Kierunki dalszych badań

- **Indukcyjne grafowe sieci neuronowe (GNN).** Naturalnym następnym krokiem po cechach topologicznych są modele uczące reprezentacji węzła z propagacją komunikatów: GraphSAGE [18], R-GCN [37] oraz HGT [21]. Metoda Node2Vec [17] została świadomie wykluczona z protokołu indukcyjnego, gdyż uczy zanurzeń *transduktywnie* (embeddingi nieporównywalne między grafami) — jest to wynik motywujący GNN, a nie luka implementacyjna.
- **Ilościowy benchmark na większym grafie zewnętrznym.** Wstępny test cross-dataset na grafie DUT (sekcja 4.9) pokazał, że detektor wytrenowany na DLG-DG-23 przenosi się na graf z innego źródła bez załamania. Graf ten był jednak bardzo mały (21 tabel) i miał łańcuchową topologię, przez co wynik ma charakter jakościowy. Naturalnym rozszerzeniem jest zbudowanie *większego* i bardziej rozgałęzionego grafu zewnętrznego (pełniejsza integracja benchmarku RDF grupy promotora [15]), który umożliwi wiarygodne porównanie ilościowe oraz ujawnienie przewagi sygnału relacyjnego, wymagającego rozgałęzień w grafie. Równie istotna jak rozmiar grafów jest ich *liczba*: przy ośmiu grafach wiarygodnych moc testów statystycznych nie wystarcza do wykrycia różnic między metodami uczonymi (sekcja 4.11).
- **Włączenie semantyki nazw i schematów.** Sygnał topologiczny badany w pracy i sygnał semantyczny (podobieństwo nazw, metadane schematu) są komplementarne [5, 8]. Jeśli dane przestaną być zanonimizowane, połączenie obu powinno przekroczyć sufit  $\approx 0,77$  właściwy podejściu czysto topologicznemu.
- **Potok dwuetapowy detekcja  $\rightarrow$  predykcja.** Wskazanie podejrzanych tabel (niniejsza praca) może służyć jako tani etap wstępny, ograniczający przestrzeń kandydatów przed kosztowną predykcją konkretnych brakujących krawędzi — łącząc oba ujęcia problemu broken lineage.
- **Rozróżnienie przyczyny zerwania.** Rozszerzenie etykiet o typ zdarzenia (UDF vs. tabela tymczasowa vs. CTE) umożliwiłoby nie tylko detekcję, lecz i diagnozę przyczyny broken lineage.

# Literatura

- [1] Leman Akoglu, Hanghang Tong, and Danai Koutra. Graph based anomaly detection and description: A survey. *Data Mining and Knowledge Discovery*, 29(3):626–688, 2015.
- [2] Witold Andrzejewski, Paweł Boiński, and Robert Wrembel. On Discovering Provenance of Data Warehouse Objects Created by Select-Project Queries. In *Proceedings of the Workshops of the EDBT/ICDT 2026 Joint Conference*, CEUR Workshop Proceedings, Tampere, Finland, 2026. CEUR-WS.org.
- [3] Apache Software Foundation. Apache Atlas: Data governance and metadata framework, 2024. <https://atlas.apache.org/>.
- [4] Albert-László Barabási and Réka Albert. Emergence of Scaling in Random Networks. *Science*, 286(5439):509–512, 1999.
- [5] Paweł Boiński, Witold Andrzejewski, Miłosz Grochowski, Tobiasz Gruszczyński, and Robert Wrembel. Leveraging machine learning techniques for discovering broken lineage links between database objects. In *Proceedings of the 33rd International Conference on Information Systems Development (ISD2025)*, Belgrade, Serbia, 2025.
- [6] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [7] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- [8] Maciej Brzeski and Adam Roman. Inferring Missing Data Lineage Links from Schema Metadata Using Transformer-Based Models. In *Proceedings of the VLDB 2025 Workshop: Applied AI for Database Systems and Applications (AIDB 2025)*. VLDB Endowment, 2025.
- [9] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Proceedings of the 8th International Conference on Database Theory (ICDT)*, pages 316–330, 2001.
- [10] Yunpeng Chen, Ying Zhao, Xuanjing Li, Jiang Zhang, Jiang Long, and Fangfang Zhou. An open dataset of data lineage graphs for data governance research. *Visual Informatics*, 8:1–5, 2024.
- [11] Yingwei Cui and Jennifer Widom. Lineage tracing for general data warehouse transformations. *The VLDB Journal*, 12(1):41–58, 2003.
- [12] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240, 2006.
- [13] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- [14] Benjamin Dietrich, Tobias Müller, and Torsten Grust. Data Provenance for Recursive SQL Queries. In *Proceedings of the 14th Workshop on the Theory and Practice of Provenance (TaPP '22)*, Philadelphia, PA, USA, 2022. ACM.

- [15] Jakub Dutkiewicz, Paweł Misiorek, and Robert Wrembel. Data Lineage Discovery in Databases Based on Knowledge Graph Link Prediction. In *Proceedings of the Workshops of the EDBT/ICDT 2026 Joint Conference*, CEUR Workshop Proceedings, Tampere, Finland, 2026. CEUR-WS.org.
- [16] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 31–40, 2007.
- [17] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 855–864, 2016.
- [18] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1024–1034, 2017.
- [19] Melanie Herschel, Ralf Diestelkämper, and Houssem Ben Lahmar. A survey on provenance: What for? what form? what from? *The VLDB Journal*, 26(6):881–906, 2017.
- [20] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [21] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference (WWW)*, pages 2704–2710, 2020.
- [22] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3146–3154, 2017.
- [23] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [24] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-01, Keele University and University of Durham, 2007.
- [25] David Kohan Marzagão, Trung Dong Huynh, Ayah Helal, Sean Baccas, and Luc Moreau. Provenance Graph Kernel. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [26] István A. Kovács, Katja Luck, Kerstin Spirohn, Yang Wang, and Albert-László Barabási. Network-based prediction of protein interactions. *Nature Communications*, 10:1240, 2019.
- [27] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV ontology. W3c recommendation, World Wide Web Consortium (W3C), 2013. <https://www.w3.org/TR/prov-o/>.
- [28] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017.
- [29] David Liben-Nowell and Jon Kleinberg. The Link-Prediction Problem for Social Networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031, 2007.
- [30] Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z. Sheng, Hui Xiong, and Leman Akoglu. A comprehensive survey on graph anomaly detection with deep learning. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12012–12038, 2023.
- [31] Víctor Martínez, Fernando Berzal, and Juan-Carlos Cubero. A survey of link prediction in complex networks. *ACM Computing Surveys*, 49(4):1–33, 2016.
- [32] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632, 2005.

- [33] OpenLineage Project. OpenLineage: An open standard for data lineage collection, 2024. Projekt Linux Foundation, <https://openlineage.io/>.
- [34] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [35] Fotis Psallidas, Ashvin Agrawal, Chandru Sugunan, Khaled Ibrahim, Konstantinos Karanasos, Jesús Camacho-Rodríguez, Avriella Floratou, Carlo Curino, and Raghu Ramakrishnan. ONEPROVENANCE: Efficient Extraction of Dynamic Coarse-Grained Provenance From Database Query Event Logs. *CoRR*, abs/2210.14047, 2022. Technical Report; deployed in Microsoft Purview.
- [36] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.
- [37] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web (ESWC)*, pages 593–607, 2018.
- [38] Chris Seiffert, Taghi M. Khoshgoftaar, Jason Van Hulse, and Amri Napolitano. RUSBoost: A hybrid approach to alleviating class imbalance. *IEEE Transactions on Systems, Man, and Cybernetics — Part A*, 40(1):185–197, 2010.
- [39] Dibin Shan, Xuehui Du, Wenjuan Wang, Na Wang, and Aodi Liu. KPI-HGNN: Key Provenance Identification Based on a Heterogeneous Graph Neural Network for Big Data Access Control. *Information Sciences*, 2023.
- [40] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- [41] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.
- [42] Masaya Yamada, Hiroyuki Kitagawa, Toshiyuki Amagasa, and Akiyoshi Matono. Augmented lineage: traceability of data analysis including complex UDF processing. *The VLDB Journal*, 32:963–983, 2023.
- [43] Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 694–699, 2002.



## Dodatek A

# Instrukcja uruchomienia i zawartość repozytorium

### A.1 Wymagania i instalacja

Implementacja wymaga interpretera Python w wersji 3.10 lub nowszej. Zależności wraz z wersjami użytymi w biegu kanonicznym wymieniono w tabeli 3.4. Środowisko przygotowuje się poleceniami:

```
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt
```

### A.2 Odtworzenie wyników rozdziału 4

Każdą tabelę rozdziału 4 można odtworzyć niezależnie. Polecenia uruchamia się z katalogu głównego repozytorium; wszystkie zapisują wyniki do katalogu `results/`.

```
# Bieg kanoniczny -- wariant uczciwy (tabela wyników głównych)
python -m src.detection.run_detection_experiments --all \
    --exclude-isolated --removal-ratio 0.10 --repeats 3 --seed 42 \
    --csv results/wyniki_detekcja_canon.csv

# Bieg kanoniczny -- wariant napompowany (kontrola przecieku izolacji)
python -m src.detection.run_detection_experiments --all \
    --removal-ratio 0.10 --repeats 3 --seed 42 \
    --csv results/wyniki_detekcja_napompowany.csv

# Testy istotności statystycznej
python scripts/test_istotnosci.py

# Badanie ablacyjne detektora
python src/detection/run_ablation.py
python scripts/test_istotnosci.py --ablacja

# Ważność cech (metoda permutacyjna)
python scripts/waznosc_cech.py
```

```
# Analiza wzraliwości na removal_ratio
python -m src.detection.run_removal_sensitivity

# Warianty protokołu -- wszystkie w konfiguracji kanonicznej
python -m src.detection.run_detection_experiments --all --single-job \
    --exclude-isolated --removal-ratio 0.10 --repeats 3 --seed 42 \
    --csv results/wyniki_jednojob.csv
python -m src.detection.run_transductive \
    --exclude-isolated --removal-ratio 0.10 --repeats 3
python -m src.detection.run_scale_generalization \
    --exclude-isolated --removal-ratio 0.10 --repeats 3

# Test cross-dataset na grafie zewnętrznym DUT
python -m src.detection.run_cross_dataset

# Tabela charakterystyki symulacji (rozdział 3)
python scripts/tabela_symulacji.py

# Tabele Dodatku B
python scripts/dodatek_wyniki.py
```

Warianty protokołu wymagają jawnego podania flag `-exclude-isolated` i `-removal-ratio 0.10`: ich wartości domyślne różnią się od biegu kanonicznego, a porównanie protokołów ma sens tylko przy identycznej konfiguracji pozostałych parametrów.

Zestaw testów jednostkowych uruchamia się poleceniem `pytest` z katalogu głównego; wszystkie 83 testy powinny przejść.

### A.3 Zawartość repozytorium

Strukturę katalogów przedstawia listing 3.1 w sekcji 3.6. Adres repozytorium podano w sekcji 3.8. Katalog `results/` zawiera pliki CSV z wynikami wszystkich biegów oraz plik `README_BIEG_KANONICZNY.md` dokumentujący konfigurację biegu, z którego pochodzą liczby raportowane w pracy.

## Dodatek B

# Pełne wyniki eksperymentów

Tabele rozdziału 4 podają wartości uśrednione po grafach wiarygodnych. Niniejszy dodatek zawiera wyniki biegu kanonicznego dla *wszystkich* 18 grafów zbioru DLG-DG-23 i wszystkich czterech metryk, wraz z grafami odrzuconymi z agregacji progiem  $\text{MIN\_POS} = 5$ . Grafy te oznaczono gwiazdką; ich metryki obliczono, lecz przy jednym lub dwóch pozytywach w zbiorze testowym są one obciążone bardzo dużą wariancją i nie powinny być interpretowane pojedynczo. Zestawienie pozwala czytelnikowi ocenić, co próg wiarygodności odsiewa i jak duży jest rozrzut wyników.

TABELA B.1: AUC-ROC dla wszystkich 18 grafów zbioru DLG-DG-23 (wyżej — lepiej). Gwiazdka przy identyfikatorze grafu — graf odrzucony z agregacji progiem  $\text{MIN\_POS} = 5$ .

| Graf           | Kand. | Zainf. | Stopień | Brzeg | Łączn. | Korzeń | Kompl. | RF    | RUSB  | LGBM  | LD*   |
|----------------|-------|--------|---------|-------|--------|--------|--------|-------|-------|-------|-------|
| DLG1*          | 13    | 1      | 0,727   | 0,583 | 0,692  | 0,792  | 0,500  | 0,545 | 0,500 | 0,591 | 0,818 |
| DLG2*          | 21    | 1      | 0,556   | 0,583 | 0,079  | 0,461  | 0,474  | 0,645 | 0,750 | 0,632 | 0,763 |
| DLG3*          | 26    | 3      | 0,142   | 0,591 | 0,295  | 0,761  | 0,477  | 0,598 | 0,644 | 0,659 | 0,674 |
| DLG4*          | 16    | 1      | 0,475   | 0,567 | 0,050  | 0,750  | 0,375  | 0,750 | 0,967 | 0,700 | 0,933 |
| DLG5           | 66    | 9      | 0,405   | 0,340 | 0,233  | 0,590  | 0,567  | 0,662 | 0,703 | 0,633 | 0,745 |
| DLG6           | 568   | 82     | 0,439   | 0,480 | 0,309  | 0,567  | 0,627  | 0,745 | 0,619 | 0,707 | 0,790 |
| DLG7*          | 29    | 1      | 0,125   | 0,339 | 0,312  | 0,179  | 0,446  | 0,464 | 0,500 | 0,625 | 0,821 |
| DLG8*          | 18    | 2      | 0,382   | 0,400 | 0,422  | 0,515  | 0,353  | 0,735 | 0,500 | 0,647 | 0,789 |
| DLG9*          | 24    | 3      | 0,217   | 0,193 | 0,222  | 0,159  | 0,205  | 0,580 | 0,591 | 0,625 | 0,517 |
| DLG10          | 213   | 36     | 0,417   | 0,472 | 0,323  | 0,544  | 0,428  | 0,681 | 0,586 | 0,588 | 0,714 |
| DLG11          | 65    | 6      | 0,513   | 0,454 | 0,386  | 0,642  | 0,492  | 0,732 | 0,658 | 0,776 | 0,804 |
| DLG12          | 197   | 26     | 0,356   | 0,469 | 0,389  | 0,457  | 0,585  | 0,855 | 0,604 | 0,837 | 0,869 |
| DLG13          | 100   | 13     | 0,446   | 0,445 | 0,198  | 0,678  | 0,602  | 0,723 | 0,674 | 0,697 | 0,692 |
| DLG14*         | 9     | 1      | 0,000   | 0,000 | 0,000  | 0,059  | 0,382  | 0,882 | 0,500 | 0,882 | 0,088 |
| DLG15*         | 25    | 2      | 0,587   | 0,450 | 0,460  | 0,380  | 0,413  | 0,580 | 0,600 | 0,670 | 0,770 |
| DLG16          | 246   | 44     | 0,451   | 0,498 | 0,366  | 0,565  | 0,626  | 0,841 | 0,599 | 0,783 | 0,818 |
| DLG17*         | 24    | 2      | 0,000   | 0,043 | 0,043  | 0,217  | 0,283  | 0,739 | 0,478 | 0,913 | 0,957 |
| DLG18          | 625   | 110    | 0,386   | 0,481 | 0,389  | 0,549  | 0,547  | 0,750 | 0,601 | 0,737 | 0,733 |
| <i>średnia</i> |       |        | 0,427   | 0,455 | 0,324  | 0,574  | 0,559  | 0,749 | 0,631 | 0,720 | 0,771 |

Skróty metod: Stopień — anomalia stopnia, Brzeg — root/leaf, Łączn. — niska łączność jobów, Korzeń — reguła korzenia, Kompl. — score kompletności, RF — Random Forest, RUSB — RUSBoost, LGBM — LightGBM, LD\* — LineageDetector (wkład własny). Średnia liczona wyłącznie po grafach wiarygodnych.

TABELA B.2: AUC-PR dla wszystkich 18 grafów zbioru DLG-DG-23 (wyżej — lepiej). Gwiazdka przy identyfikatorze grafu — graf odrzucony z agregacji progiem MIN\_POS = 5.

| Graf           | Kand. | Zainf. | Stopień | Brzeg | Łączn. | Korzeń | Kompl. | RF    | RUSB  | LGBM  | LD*   |
|----------------|-------|--------|---------|-------|--------|--------|--------|-------|-------|-------|-------|
| DLG1*          | 13    | 1      | 0,310   | 0,091 | 0,125  | 0,167  | 0,077  | 0,225 | 0,077 | 0,250 | 0,450 |
| DLG2*          | 21    | 1      | 0,139   | 0,077 | 0,074  | 0,089  | 0,095  | 0,562 | 0,548 | 0,562 | 0,591 |
| DLG3*          | 26    | 3      | 0,111   | 0,182 | 0,121  | 0,320  | 0,154  | 0,457 | 0,365 | 0,404 | 0,588 |
| DLG4*          | 16    | 1      | 0,083   | 0,056 | 0,050  | 0,091  | 0,062  | 0,167 | 0,500 | 0,143 | 0,500 |
| DLG5           | 66    | 9      | 0,284   | 0,127 | 0,097  | 0,168  | 0,258  | 0,373 | 0,323 | 0,460 | 0,469 |
| DLG6           | 568   | 82     | 0,140   | 0,140 | 0,105  | 0,170  | 0,229  | 0,378 | 0,234 | 0,354 | 0,427 |
| DLG7*          | 29    | 1      | 0,038   | 0,043 | 0,045  | 0,034  | 0,034  | 0,062 | 0,034 | 0,130 | 0,167 |
| DLG8*          | 18    | 2      | 0,083   | 0,086 | 0,085  | 0,108  | 0,105  | 0,250 | 0,167 | 0,562 | 0,567 |
| DLG9*          | 24    | 3      | 0,096   | 0,115 | 0,095  | 0,115  | 0,115  | 0,217 | 0,115 | 0,248 | 0,200 |
| DLG10          | 213   | 36     | 0,177   | 0,168 | 0,129  | 0,195  | 0,188  | 0,382 | 0,246 | 0,370 | 0,397 |
| DLG11          | 65    | 6      | 0,132   | 0,100 | 0,081  | 0,150  | 0,108  | 0,355 | 0,279 | 0,327 | 0,368 |
| DLG12          | 197   | 26     | 0,125   | 0,111 | 0,103  | 0,115  | 0,219  | 0,478 | 0,236 | 0,342 | 0,474 |
| DLG13          | 100   | 13     | 0,120   | 0,110 | 0,081  | 0,180  | 0,129  | 0,349 | 0,259 | 0,300 | 0,380 |
| DLG14*         | 9     | 1      | 0,056   | 0,056 | 0,056  | 0,056  | 0,056  | 0,333 | 0,056 | 0,333 | 0,059 |
| DLG15*         | 25    | 2      | 0,084   | 0,062 | 0,068  | 0,068  | 0,080  | 0,262 | 0,360 | 0,302 | 0,512 |
| DLG16          | 246   | 44     | 0,201   | 0,166 | 0,143  | 0,163  | 0,301  | 0,613 | 0,196 | 0,506 | 0,518 |
| DLG17*         | 24    | 2      | 0,042   | 0,042 | 0,043  | 0,042  | 0,042  | 0,200 | 0,042 | 0,333 | 0,500 |
| DLG18          | 625   | 110    | 0,143   | 0,177 | 0,155  | 0,199  | 0,218  | 0,472 | 0,223 | 0,393 | 0,448 |
| <i>średnia</i> |       |        | 0,165   | 0,137 | 0,112  | 0,168  | 0,206  | 0,425 | 0,249 | 0,381 | 0,435 |

Skróty metod: Stopień — anomalia stopnia, Brzeg — root/leaf, Łączn. — niska łączność jobów, Korzeń — reguła korzenia, Kompl. — score kompletności, RF — Random Forest, RUSB — RUSBoost, LGBM — LightGBM, LD\* — LineageDetector (wkład własny). Średnia liczona wyłącznie po grafach wiarygodnych.

TABELA B.3: Precision@k dla wszystkich 18 grafów zbioru DLG-DG-23 (wyżej — lepiej). Gwiazdka przy identyfikatorze grafu — graf odrzucony z agregacji progiem MIN\_POS = 5.

| Graf           | Kand. | Zainf. | Stopień | Brzeg | Łączn. | Korzeń | Kompl. | RF    | RUSB  | LGBM  | LD*   |
|----------------|-------|--------|---------|-------|--------|--------|--------|-------|-------|-------|-------|
| DLG1*          | 13    | 1      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,000 | 0,000 | 0,000 | 0,000 |
| DLG2*          | 21    | 1      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,500 | 0,500 | 0,500 | 0,500 |
| DLG3*          | 26    | 3      | 0,000   | 0,000 | 0,250  | 0,250  | 0,000  | 0,500 | 0,333 | 0,500 | 0,500 |
| DLG4*          | 16    | 1      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,000 | 0,000 | 0,000 | 0,000 |
| DLG5           | 66    | 9      | 0,273   | 0,111 | 0,000  | 0,444  | 0,273  | 0,364 | 0,444 | 0,444 | 0,444 |
| DLG6           | 568   | 82     | 0,138   | 0,138 | 0,068  | 0,191  | 0,311  | 0,340 | 0,338 | 0,351 | 0,432 |
| DLG7*          | 29    | 1      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,000 | 0,000 | 0,000 | 0,000 |
| DLG8*          | 18    | 2      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,000 | 0,000 | 0,500 | 0,333 |
| DLG9*          | 24    | 3      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,000 | 0,000 | 0,000 | 0,000 |
| DLG10          | 213   | 36     | 0,140   | 0,133 | 0,056  | 0,139  | 0,070  | 0,349 | 0,349 | 0,361 | 0,395 |
| DLG11          | 65    | 6      | 0,143   | 0,167 | 0,167  | 0,000  | 0,333  | 0,167 | 0,333 | 0,333 | 0,333 |
| DLG12          | 197   | 26     | 0,182   | 0,080 | 0,080  | 0,080  | 0,280  | 0,520 | 0,273 | 0,406 | 0,480 |
| DLG13          | 100   | 13     | 0,000   | 0,167 | 0,083  | 0,176  | 0,000  | 0,250 | 0,417 | 0,333 | 0,417 |
| DLG14*         | 9     | 1      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,000 | 0,000 | 0,000 | 0,000 |
| DLG15*         | 25    | 2      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,200 | 0,200 | 0,200 | 0,600 |
| DLG16          | 246   | 44     | 0,146   | 0,214 | 0,179  | 0,195  | 0,400  | 0,600 | 0,317 | 0,500 | 0,585 |
| DLG17*         | 24    | 2      | 0,000   | 0,000 | 0,000  | 0,000  | 0,000  | 0,000 | 0,000 | 0,000 | 0,250 |
| DLG18          | 625   | 110    | 0,117   | 0,157 | 0,117  | 0,214  | 0,252  | 0,447 | 0,252 | 0,456 | 0,456 |
| <i>średnia</i> |       |        | 0,142   | 0,146 | 0,094  | 0,180  | 0,240  | 0,380 | 0,340 | 0,398 | 0,443 |

Skróty metod: Stopień — anomalia stopnia, Brzeg — root/leaf, Łączn. — niska łączność jobów, Korzeń — reguła korzenia, Kompl. — score kompletności, RF — Random Forest, RUSB — RUSBoost, LGBM — LightGBM, LD\* — LineageDetector (wkład własny). Średnia liczona wyłącznie po grafach wiarygodnych.

TABELA B.4: Brier score dla wszystkich 18 grafów zbioru DLG-DG-23 (niżej — lepiej). Gwiazdka przy identyfikatorze grafu — graf odrzucony z agregacji progiem MIN\_POS = 5.

| Graf           | Kand. | Zainf. | Stopień | Brzeg | Łączn. | Korzeń | Kompl. | RF    | RUSB  | LGBM  | LD*   |
|----------------|-------|--------|---------|-------|--------|--------|--------|-------|-------|-------|-------|
| DLG1*          | 13    | 1      | 0,071   | 0,769 | 0,192  | 0,385  | 0,077  | 0,129 | 0,123 | 0,181 | 0,109 |
| DLG2*          | 21    | 1      | 0,086   | 0,750 | 0,228  | 0,571  | 0,100  | 0,106 | 0,050 | 0,158 | 0,073 |
| DLG3*          | 26    | 3      | 0,146   | 0,692 | 0,209  | 0,231  | 0,192  | 0,127 | 0,102 | 0,163 | 0,108 |
| DLG4*          | 16    | 1      | 0,058   | 0,812 | 0,245  | 0,476  | 0,107  | 0,090 | 0,050 | 0,153 | 0,054 |
| DLG5           | 66    | 9      | 0,132   | 0,815 | 0,243  | 0,385  | 0,133  | 0,146 | 0,155 | 0,171 | 0,115 |
| DLG6           | 568   | 82     | 0,133   | 0,694 | 0,221  | 0,460  | 0,141  | 0,115 | 0,151 | 0,141 | 0,099 |
| DLG7*          | 29    | 1      | 0,038   | 0,793 | 0,227  | 0,655  | 0,060  | 0,140 | 0,065 | 0,395 | 0,050 |
| DLG8*          | 18    | 2      | 0,106   | 0,778 | 0,244  | 0,474  | 0,194  | 0,124 | 0,099 | 0,167 | 0,078 |
| DLG9*          | 24    | 3      | 0,116   | 0,846 | 0,228  | 0,731  | 0,234  | 0,178 | 0,161 | 0,214 | 0,105 |
| DLG10          | 213   | 36     | 0,166   | 0,759 | 0,231  | 0,553  | 0,185  | 0,162 | 0,156 | 0,161 | 0,136 |
| DLG11          | 65    | 6      | 0,088   | 0,862 | 0,247  | 0,415  | 0,108  | 0,097 | 0,075 | 0,122 | 0,070 |
| DLG12          | 197   | 26     | 0,117   | 0,732 | 0,204  | 0,611  | 0,123  | 0,097 | 0,131 | 0,131 | 0,089 |
| DLG13          | 100   | 13     | 0,112   | 0,850 | 0,242  | 0,440  | 0,126  | 0,113 | 0,113 | 0,180 | 0,106 |
| DLG14*         | 9     | 1      | 0,054   | 1,000 | 0,261  | 0,889  | 0,130  | 0,151 | 0,086 | 0,072 | 0,086 |
| DLG15*         | 25    | 2      | 0,082   | 0,875 | 0,236  | 0,720  | 0,133  | 0,135 | 0,078 | 0,162 | 0,086 |
| DLG16          | 246   | 44     | 0,149   | 0,605 | 0,208  | 0,419  | 0,158  | 0,085 | 0,188 | 0,117 | 0,105 |
| DLG17*         | 24    | 2      | 0,046   | 0,917 | 0,239  | 0,583  | 0,176  | 0,086 | 0,078 | 0,198 | 0,046 |
| DLG18          | 625   | 110    | 0,170   | 0,643 | 0,215  | 0,470  | 0,181  | 0,125 | 0,249 | 0,156 | 0,128 |
| <i>średnia</i> |       |        | 0,134   | 0,745 | 0,226  | 0,469  | 0,144  | 0,117 | 0,152 | 0,147 | 0,106 |

Skróty metod: Stopień — anomalia stopnia, Brzeg — root/leaf, Łączn. — niska łączność jobów, Korzeń — reguła korzenia, Kompl. — score kompletności, RF — Random Forest, RUSB — RUSBoost, LGBM — LightGBM, LD\* — LineageDetector (wkład własny). Średnia liczona wyłącznie po grafach wiarygodnych.

## Dodatek C

# Przegląd konfiguracji testu cross-dataset

Sekcja 4.9 raportuje wynik na zewnętrznym grafie DUT dla jednej konfiguracji. Aby sprawdzić, czy wnioski nie zależą od jej doboru, test powtórzono dla dwóch wariantów izolacji i trzech wartości parametru `removal_ratio`, po 25 powtórzeniach na konfigurację. Tabela C.1 zestawia AUC-ROC; pełne wyniki, obejmujące pozostałe metryki, zawiera plik `results/cross_dataset_dut_sweep.md` w repozytorium.

TABELA C.1: AUC-ROC na zewnętrznym grafie DUT w zależności od wariantu izolacji i parametru `removal_ratio` (trening na małych i średnich grafach DLG-DG-23, 25 powtórzeń na konfigurację).

| Metoda                           | Wariant napompowany |       |       | Wariant uczciwy |       |       |
|----------------------------------|---------------------|-------|-------|-----------------|-------|-------|
|                                  | 0,2                 | 0,3   | 0,4   | 0,2             | 0,3   | 0,4   |
| Anomalia stopnia                 | 0,851               | 0,866 | 0,881 | 0,766           | 0,762 | 0,748 |
| Brzeg (root/leaf)                | 0,828               | 0,847 | 0,873 | 0,726           | 0,725 | 0,727 |
| Niska łączność zadań             | 0,851               | 0,866 | 0,881 | 0,766           | 0,762 | 0,748 |
| Score kompletności               | 0,500               | 0,500 | 0,500 | 0,500           | 0,500 | 0,500 |
| Reguła korzenia                  | 0,705               | 0,723 | 0,779 | 0,630           | 0,622 | 0,686 |
| Random Forest                    | 0,758               | 0,793 | 0,847 | 0,646           | 0,618 | 0,680 |
| RUSBoost                         | 0,627               | 0,681 | 0,765 | 0,484           | 0,483 | 0,504 |
| LightGBM                         | 0,794               | 0,813 | 0,845 | 0,663           | 0,664 | 0,656 |
| LineageDetector*                 | 0,767               | 0,812 | 0,843 | 0,654           | 0,646 | 0,684 |
| <i>śr. liczba zainfekowanych</i> | 6,6                 | 8,5   | 12,2  | 3,9             | 4,5   | 5,4   |

Zwraca uwagę wiersz score’u kompletności: wartość 0,500 powtarza się we wszystkich sześciu konfiguracjach, co potwierdza, że sygnał peer-consistency jest na tym grafie tożsamościowo zerowy z przyczyn strukturalnych — łańcuchowa topologia nie daje tabelom peerów — a nie przypadkowo słaby. Ostatni wiersz wyjaśnia z kolei, dlaczego wariant uczciwy wypada gorzej od napompowanego: przy 21 tabelach-kandydatach liczba pozytywów spada do czterech, czyli poniżej progu wiarygodności przyjętego dla DLG-DG-23.



© 2026 Maria Nowicka

Instytut Informatyki, Wydział Informatyki i Telekomunikacji  
Politechnika Poznańska

Skład przy użyciu systemu  $\text{\LaTeX}$  na platformie Overleaf.